BANGALORE INSTITUTE OF TECHNOLOGY

K. R Road, V. V. Pura, Bengaluru, Karnataka, India – 560004
Affiliated to VTU, approved by AICTE and UGC, Accredited by NAAC with A+ and NBA

DEPARTMENT OF MATHEMATICS

LAB MANUAL

2023-2024

Lab Component of First Semester B.E.,
Mathematics-I
for
Mechanical Engineering Stream
BMATM101

Name:……

USN:……...

Branch:…….....Section:..……..Batch:.....

Mobile:……...

Email:……

BANGALORE INSTITUTE OF TECHNOLOGY

Vision

Establish and develop the Institute as the Centre of higher learning, ever abreast with expanding horizon of knowledge in the field of Engineering and Technology with entrepreneurial thinking, leadership excellence for life-long success and solve societal problems.

Mission

- Provide high quality education in the Engineering disciplines from the undergraduate through doctoral levels with creative academic and professional programs.
- Develop the Institute as a leader in Science, Engineering, Technology, Management and Research and apply knowledge for the benefit of society.
- Establish mutual beneficial partnerships with Industry, Alumni, Local, State and Central Governments by Public Service Assistance and Collaborative Research.
- Inculcate personality development through sports, cultural and extracurricular activities and engage in social, economic and professional challenges.

DEPARTMENT OF MATHEMATICS

Vision

To provide with sufficient understanding and educational experience in all areas of Mathematics.

Mission

- Providing quality education through innovative teaching.
- Applying mathematics as a tool to model and solve Engineering problems.
- Creating a pool of talent to undertake research and development.

General Instructions:

1. Sign in the log book when you enter/leave the laboratory.
2. Records have to be submitted every week for evaluation.
3. The program to be executed in the respective lab session has to be written in the lab observation copy beforehand.
4. Attendance in all the labs is mandatory, absence is permitted only with prior permission from the Class teacher
5. Every student should know the location and operating procedures of all Safety equipment including the First Aid Kit and Fire extinguisher.
6. Do not open any irrelevant websites in labs
7. The workplace has to be tidy before, during, and after the experiment, and do not eat food, drink beverages, or chew gum in the laboratory.

Method of evaluation and rubrics:

- Each Lab is conducted for 15 marks and CIE practical marks will be the average marks of all the lab sessions.

LAB RECORD	Submission of record with the complete write-up on time	5 marks (for each lab)
LAB Session	Execution of programs from the Exercise section in the manual	10 marks (For each lab)
Eligibility: A minimum of 06 marks should be scored out of 15 marks to take up the theory Semester End Exam.		

- The laboratory test at the end of the 15th week of the semester/after completion of all the experiments (whichever is early) shall be conducted for 50 marks and scaled down to 10 marks. A minimum of 04 marks should be scored out of 10 marks to take up the theory Semester End Examination.
- The laboratory component shall be for CIE only. However, in SEE, the questions from the laboratory component shall be included.

CONTENTS

Sl. No.	Topic
1	LAB 1: 2D plots of Cartesian and Polar curves
2	LAB 2: Finding the angle between two polar curves, curvature, and radius of curvature both in polar and parametric form
3	LAB 3: Finding Partial derivatives and Jacobian of functions of several variables
4	LAB 4: Finding Maclaurin's series expansion, Limit of a function, and Maxima and Minima of functions of two variables
5	LAB 5: Solving first-order ODEs, plotting solutions of IVPs and Application problems
6	LAB 6: Solution of second-order ordinary differential equation and plotting the solution curve
7	LAB 7: Solution of differential equation Oscillations of a spring with various load
8	LAB 8: Determining whether the given homogeneous linear system has a non-trivial solution and to solve non non-homogeneous linear system if it is consistent
9	LAB 9: Solution of a system of linear non-homogeneous equations using the Gauss-Seidel method
10	LAB 10: Determining the eigenvalues and eigenvectors for the given matrix and finding the numerically largest eigenvalue by using the Rayleigh Power method

LAB 1: 2D plots of Cartesian and Polar curves

Python modules used in the lab:

```
plot(x, y)         # plot x and y using default line style and color
plot(x, y, 'bo')  # plot x and y using blue circle markers
plot(y)           # plot y using x as index array 0..N-1
plot(y, 'r+')     # ditto, but with red plusses
plot(x, y, 'go--', linewidth=2, markersize=12)
plot(x, y, color='green', marker='o', linestyle='dashed', linewidth=2,
      markersize=12)
scatter(x, y)    # plot scatter graph of y vs x
polar(theta, r) # make a polar plot of r vs theta
title(label, fontdict=None, loc=None, pad=None, y=None) #Set a title for
                                                          the Axes
xlabel(xlabel, fontdict=None, labelpad=None, loc=None) #set a title for the
                                                           x-axis
ylabel(ylabel, fontdict=None, labelpad=None, loc=None) #set a title for the
                                                          y-axis
grid(visible=None, which='major', axis='both') # configure the grid lines
axvline(x=0, ymin=0, ymax=1) # add a vertical line across the Axis
axhline(y=0, xmin=0, xmax=1)  #add a horizontal line across the Axis
show(close=None, block=None) # display all open figures
legend()         # place a legend on the Axes
linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None,
    axis=0)    # Return evenly spaced numbers over a specified interval
```

```
plot_implicit(expr, x_var=None, y_var=None, adaptive=True, depth=0,
     points=300, line_color='blue', show=True) # plot an implicit function
Eq(lhs, rhs=None, evaluate=False)  # An equal relation between two objects
x, y, z = symbols('x,y,z')  # to define symbols x,y,z
a, b, c = symbols('a b c')  #to define a,b,c
x,y = symbols('x y', real=True, positive=True) #here defined symbols are
                                                 real and positive
x = Symbol('x') #to define single symbol
```

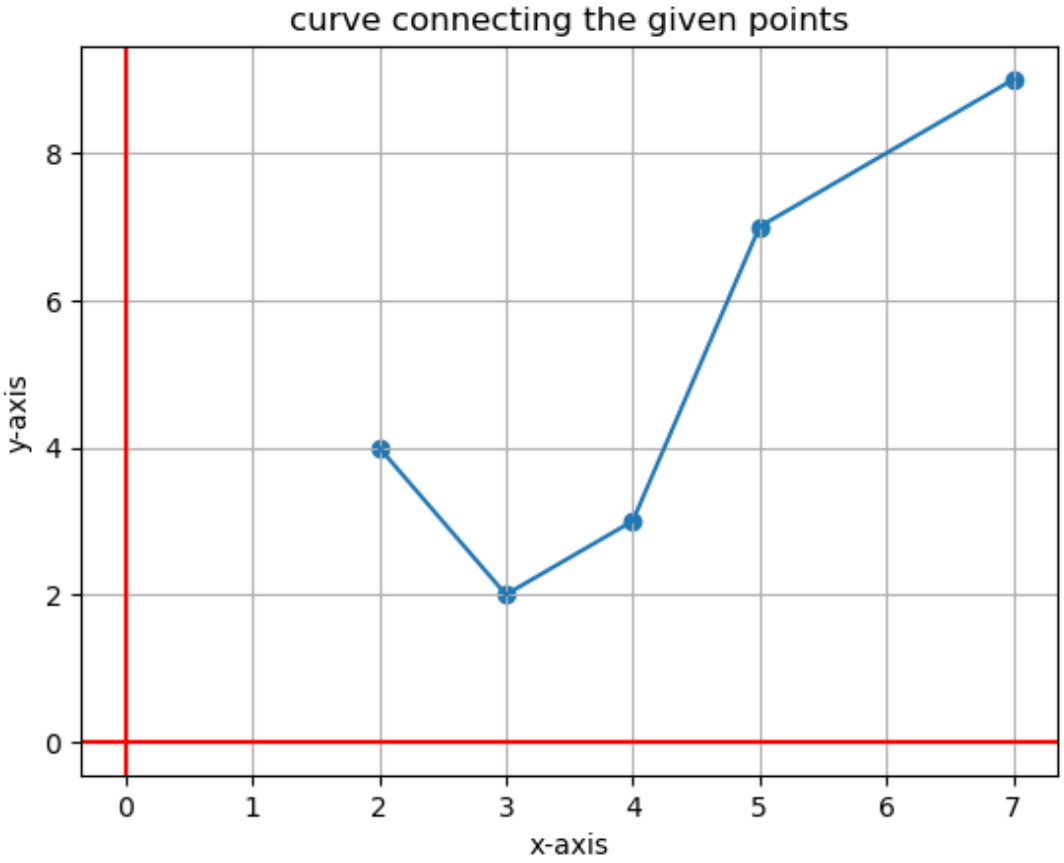
Program 1: Program to plot curve connecting following points using plot and scatter modules

x: 2 3 4 5 7

y: 4 2 3 7 9

```
from pylab import *
x=[2,3,4,5,7]
y=[4,2,3,7,9]
if len(x)!=len(y):
    print("number of values of x and y doesnot match")
else:
    scatter(x,y)
    plot(x,y)
    grid()
    xlabel("x-axis")
    ylabel("y-axis")
    title("curve connecting the given points")
    axvline(x=0,color="red")
    axhline(y=0,color="red")
    show()
```

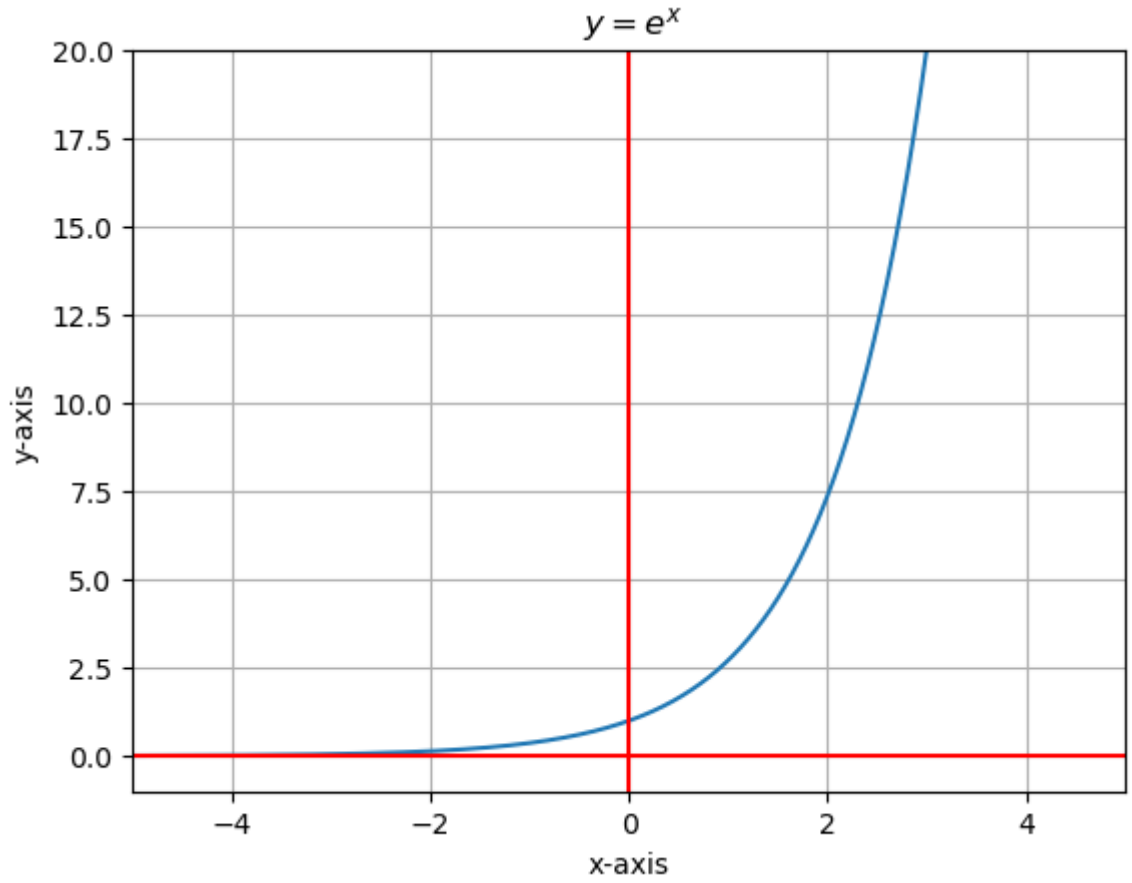
Output:



Program 2: Program to plot exponential function $y = e^x$

```
from pylab import *
x = arange(-5,5,0.001)
y = exp(x)
plot(x,y)
title(" $ y=e^x $")
grid()
xlabel("x-axis")
ylabel("y-axis")
xlim(-5,5)
ylim(-1,20)
axvline(x=0,color="red")
axhline(y=0,color="red")
show()
```

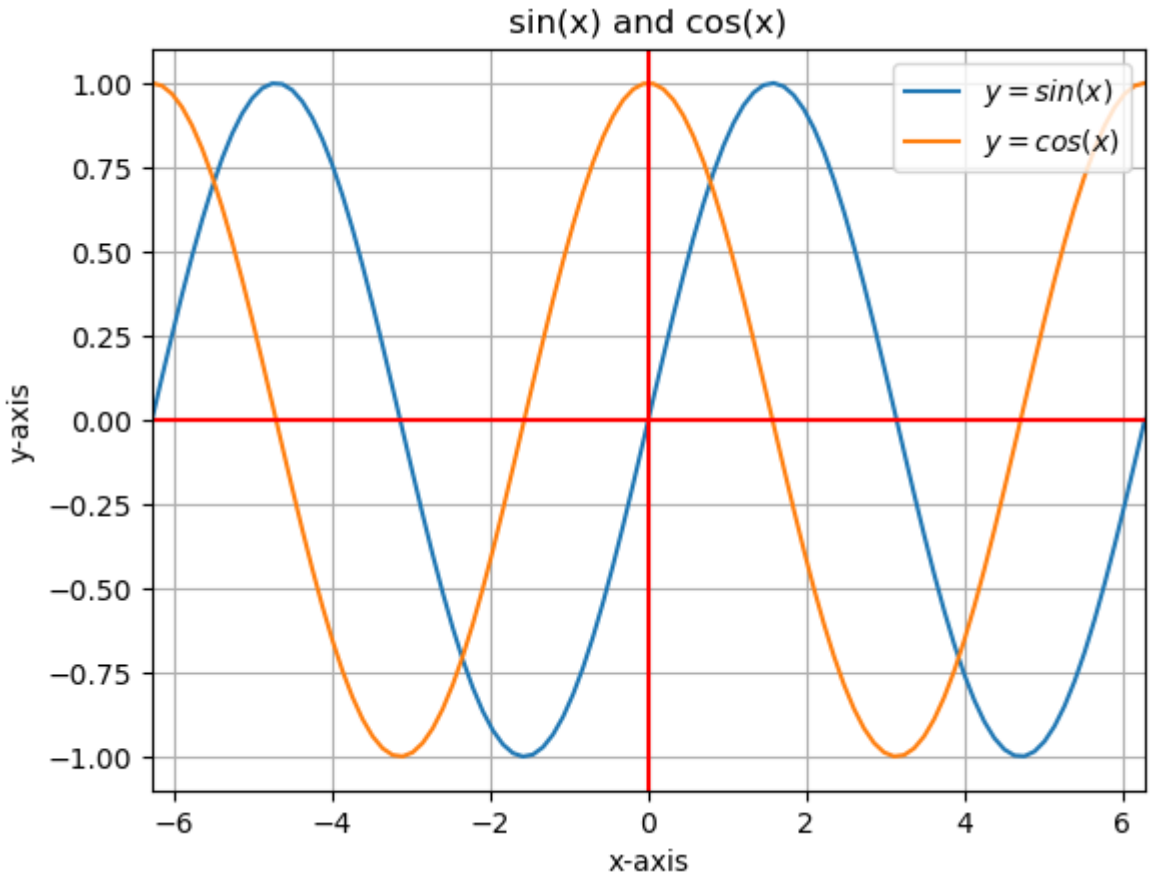
Output:



Program 3: Program to plot sin(x) and cos(x) together

```
from pylab import *
x = linspace(-2*pi,2*pi,100)
plot(x,sin(x),label="$y=sin(x)$")
plot(x,cos(x),label="$y=cos(x)$")
title(" Exponential curve ")
grid()
xlabel("x-axis")
ylabel("y-axis")
title("sin(x) and cos(x)")
legend()
xlim(-2*pi,2*pi)
axvline(color="red")
axhline(color="red")
show()
```

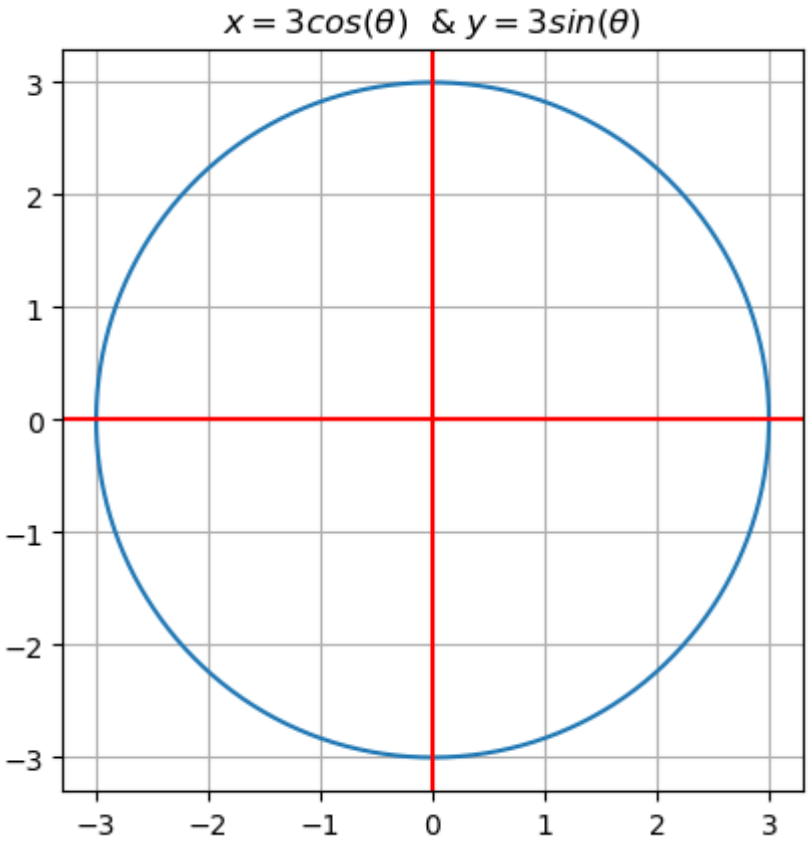
Output:



Program 4: Program to plot parametric curve, circle $x = r\cos(\theta)$ and $y = r\sin(\theta)$

```
from pylab import *
theta=linspace(0,2*pi,1000)
x=3*cos(theta)
y=3*sin(theta)
plot(x,y)
axis("square")
title("$x=3 cos(\\theta)$  & $ y=3 sin(\\theta)$")
axvline(color="red")
axhline(color="red")
grid()
show()
```

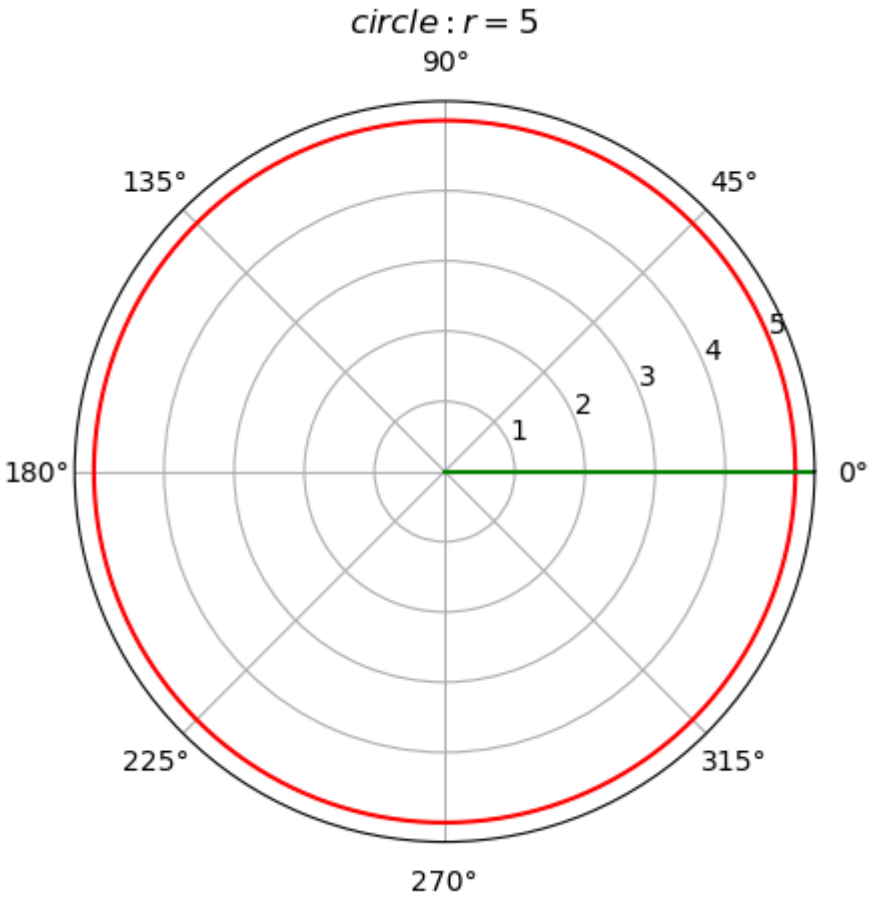
Output:



Program 5: Program to plot circle $r = 5$

```
from pylab import *
theta=linspace(0,2*pi,1000)
r=[5 for i in theta]
polar(theta,r,color="red")
title("$circle: r= 5 $")
axvline(color="green")
show()
```

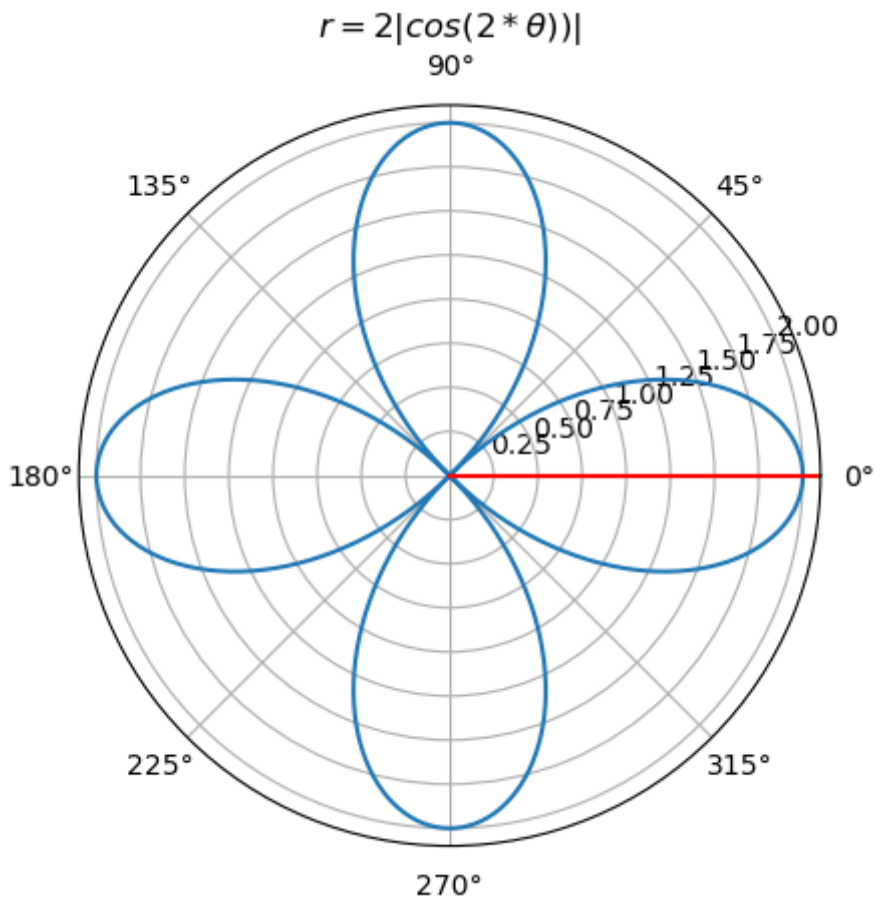
Output:



Program 6: Program to plot four leaved rose $r = 2\left|\cos(2\theta)\right|$

```
1 from pylab import *
2 theta=linspace(0,2*pi,1000)
3 r=2*abs(cos(2*theta))
4 polar(theta,r)
5 title("$r=2|cos(2*\\theta))|$")
6 axvline(color="red")
7 show()
```

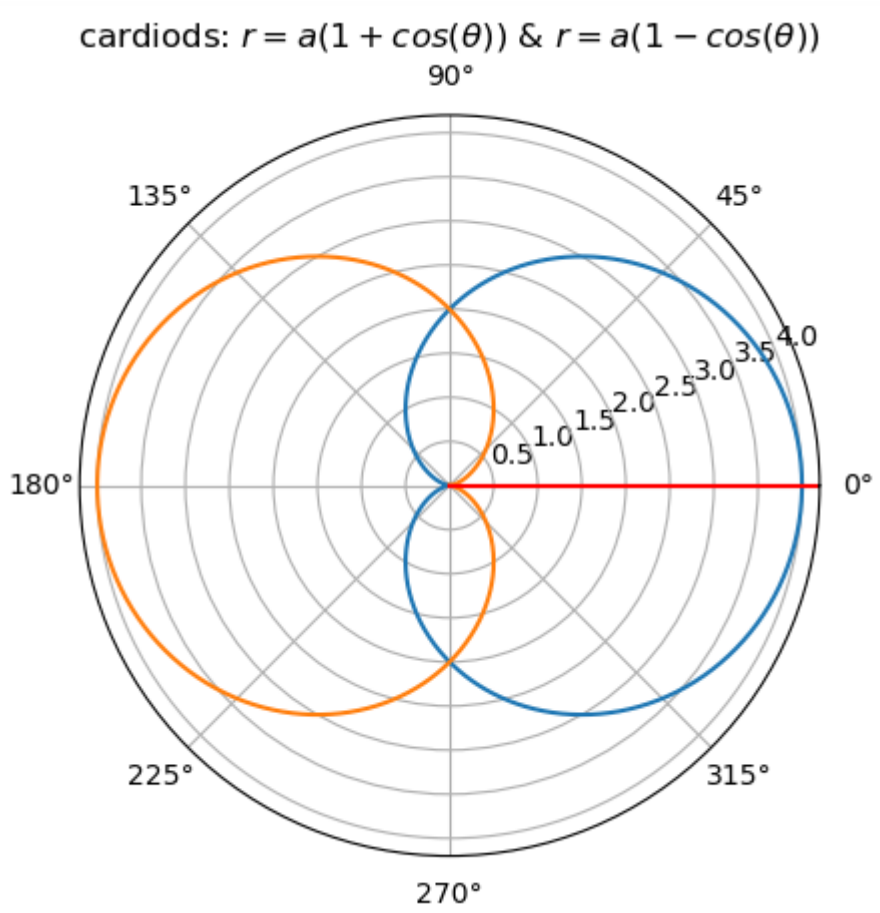
Output:



Program 7: Program to plot cardioids $r = a\left(1+\cos\left(\theta\right)\right)$ and $r = a\left(1-\cos\left(\theta\right)\right)$ taking $a = 2$.

```
from pylab import *
theta=linspace(0,2*pi,1000)
r1=2*(1+cos(theta))
r2=2*(1-cos(theta))
polar(theta,r1,theta,r2)
title("cardiods: $r=a(1+cos(\\theta))$ & $r=a(1-cos(\\theta))$")
axvline(color="red")
show()
```

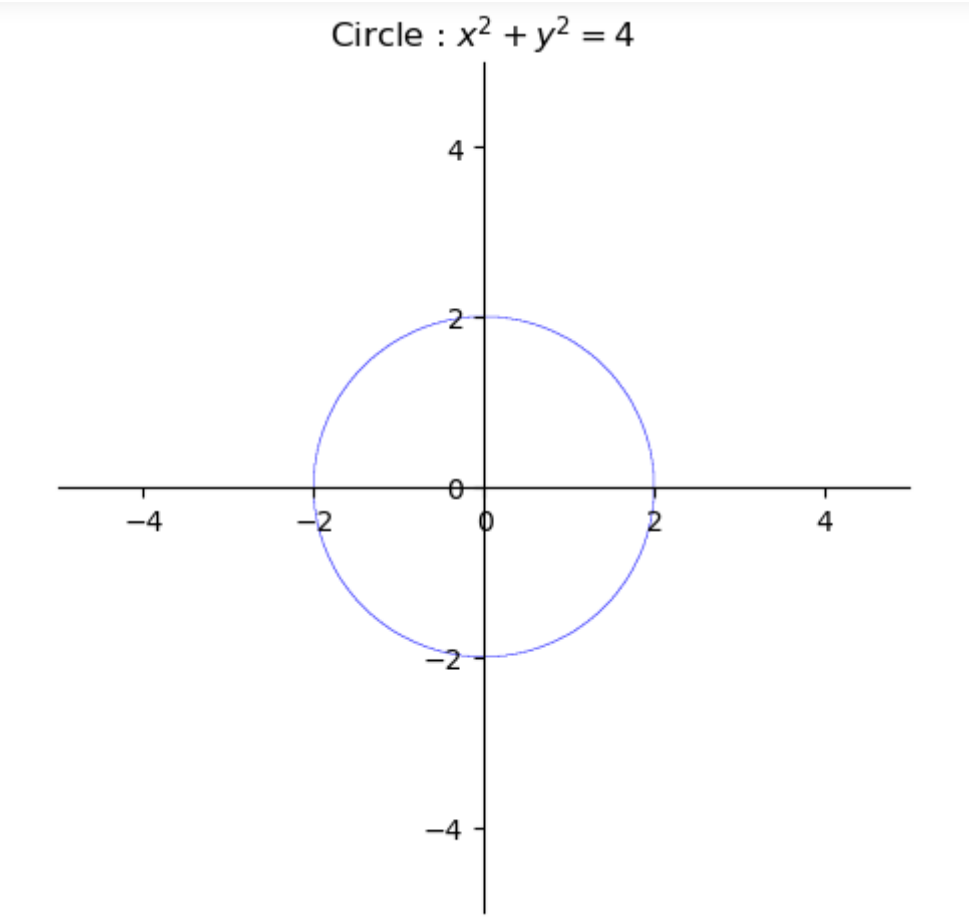
Output:



Program 8: Program to plot implicit function, Circle: $x^2 + y^2 = a^2$; take $a = 2$

```
from pylab import *
from sympy import *
x,y=symbols('x,y')
p1=plot_implicit(Eq(x**2 + y**2,4),xlabel=False,ylabel=False,
                 title='Circle : $x^2+y^2=4$ ',aspect_ratio=(2.,2.)) # a= 2
show()
```

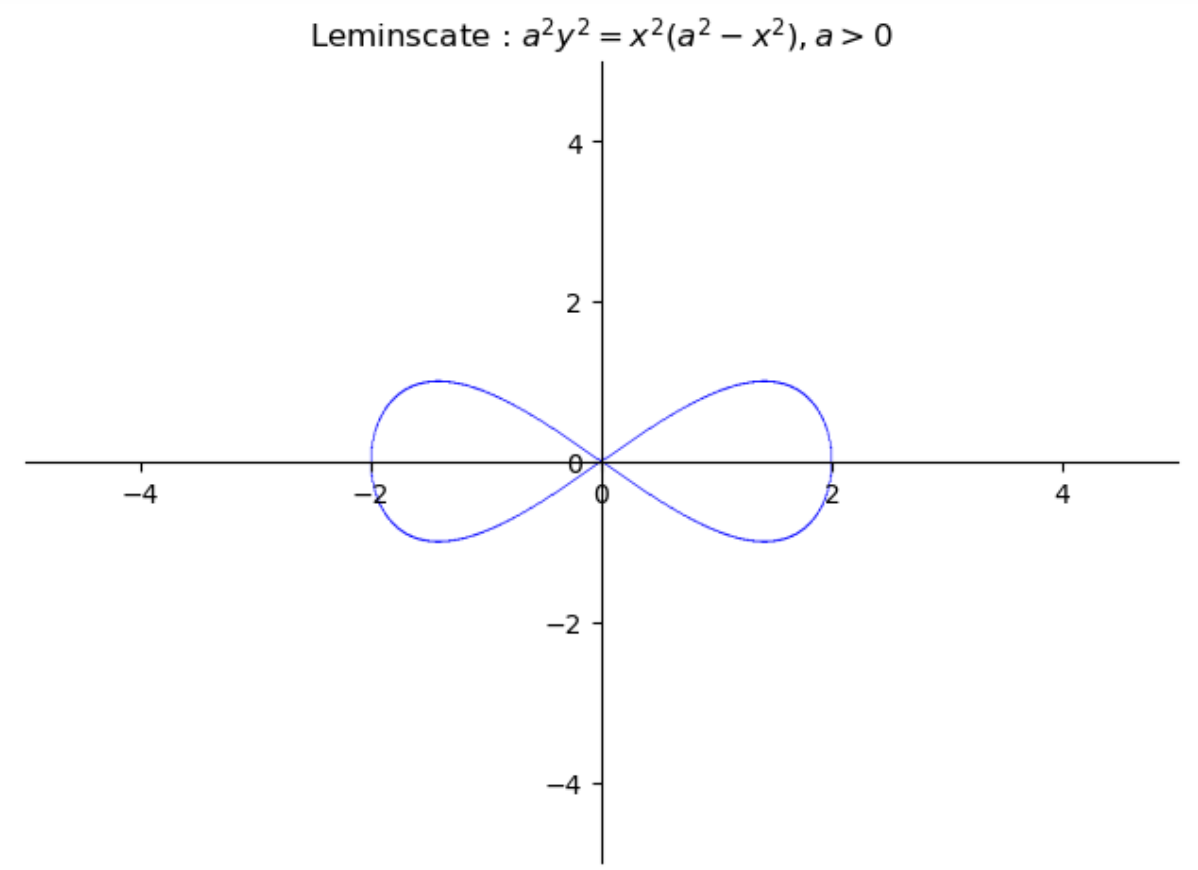
Output:



Program 9: Program to plot implicit function, Lemniscate: $a^2y^2 = x^2\left(a^2 - x^2\right)$; take $a = 2$

```
from sympy import *
from pylab import *
x,y=symbols('x y')
p=plot_implicit(Eq(4*(y**2),(x**2)*(4-x**2)),xlabel=False,ylabel=False,
                title='Leminscate : $a^2 y^2 = x^2 (a^2-x^2), a> 0$ ') # a= 2
show()
```

Output:



Exercise: Write python program to plot the following

1. Curve connecting following points using plot and scatter modules

x:	1	3	5	7	9
y:	2	4	6	8	10

2. Linear curve $y = x$, quadratic curve $y = x^2$ and cubic curve $y = x^3$ together

3. *Catenary* $y = a\cosh\left(\frac{x}{a}\right)$; take $a = 2$

4. Parametric curve, *Cycliod* $x = r\left(\theta - \sin\left(\theta\right)\right)$ and $y = r\left(1 - \cos\left(\theta\right)\right)$, take $r = 2$ and θ from 0 to 6*pi

5. Parametric curve, *Lissajous* curves $x = 2\sin\left(3\theta + \pi/3\right)$ and $y = \sin\left(\theta\right)$

6. Polar curve *ellipse* $r = \dfrac{a*b}{\sqrt{a\sin^2\theta + b\cos^2\theta}}$, take a = 4 and b = 2

7. Polar curve *cardioid* $r = a\left(1 - \sin\left(\theta\right)\right)$, take a = 2

8. Polar curve *spiral of Archimedes* $r = a + b\theta$, take a = 2 and take b = 3

9. Implicit function, *Strophoid*: $\left(a - x\right)y^2 = x^2\left(a + x\right)$; take a = 2

10. Implicit function, *Cissoid*: $\left(a - x\right)y^2 = x^3$; take a = 2

11. Implicit function, *Folium of De-cartes*: $x^3 + y^3 = 3axy$; take a = 2

12. Implicit function, *Astroid*: $x^{2/3} + y^{2/3} = a^{2/3}$; take a = 4

13. Implicit function, *hyperbola*: $\dfrac{x^2}{a^2} - \dfrac{y^2}{b^2} = 1$; take a = 4 and b = 3

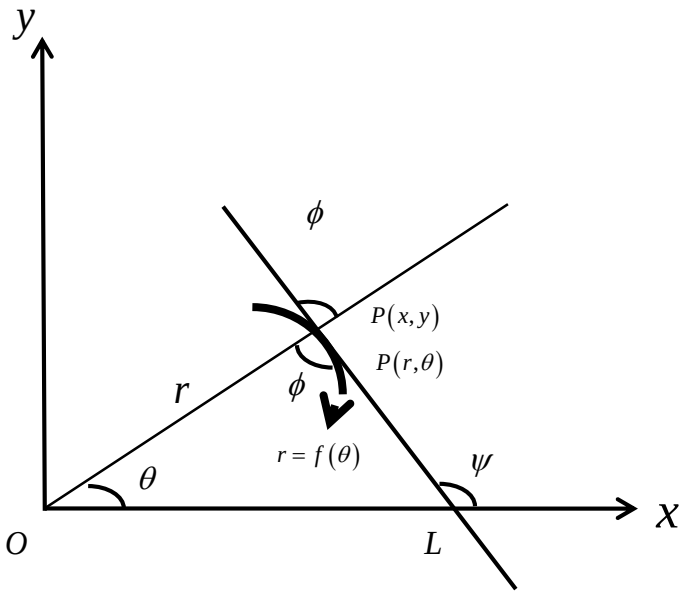
OBSERVATIONS

LAB 2: Finding the angle between two polar curves, curvature, and radius of curvature both in polar and parametric form

Angle between radius vector and tangent:

Let $P=(r,\theta)$ be a point on the polar curve and PT be the tangent at P making an angle ψ with x axis. If ϕ is the angle between radius vector and the tangent then one can show

$$\phi=\tan^{-1}\left(r\Big/\frac{dr}{d\theta}\right)$$



Example:

Find the angle between the radius vector and the tangent for the curve $r=a(1-\cos\theta)$

Solution: Consider $r=a(1-\cos\theta)$

$$\frac{dr}{d\theta}=a\sin\theta$$

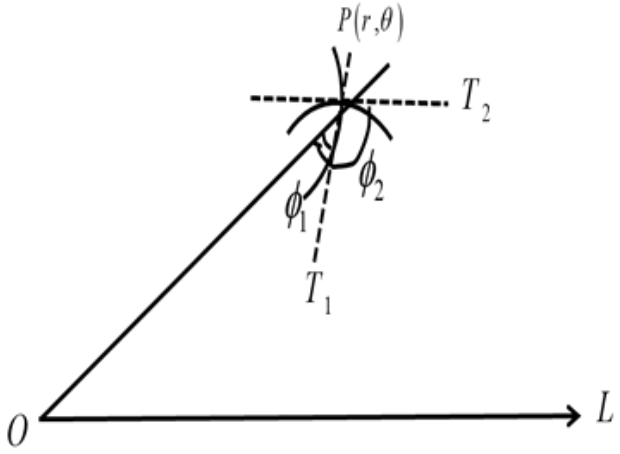
$$\Rightarrow\tan\phi=\left(r\Big/\frac{dr}{d\theta}\right)=\frac{1-\cos\theta}{\sin\theta}=\tan\left(\frac{\theta}{2}\right)$$

$$\Rightarrow\tan\phi=\tan\left(\frac{\theta}{2}\right)$$

$$\Rightarrow\phi=\frac{\theta}{2}$$

Angle between polar curves:

Let C_1 and C_2 be two curves interesting at P. The angle between C_1 and C_2 is the angle between tangents PT_1 and PT_2. If ϕ_1 is the angle between radius vector and PT_1 and ϕ_2 is the angle between radius vector and PT_2, then angle between C_1 and C_2 is $|\phi_1-\phi_2|$.



Working rule to find the angle between two polar curves $r = f(t)$ and $r = g(t)$

Step1: Solve the curves to find the value of t at the point of intersection, let it be θ.

Step2: Find $\tan\phi_1 = \left(r \Big/ \frac{dr}{dt} \right)$ for curve $r = f(t)$ and $\tan\phi_2 = \left(r \Big/ \frac{dr}{dt} \right)$ for curve $r = g(t)$

Step 3: Substitute $t = \theta$ in $\tan\phi_1 = \left(r \Big/ \frac{dr}{dt} \right)$ and $\tan\phi_2 = \left(r \Big/ \frac{dr}{dt} \right)$

Step 4: Compute $\phi_1 = \tan^{-1}\left(r \Big/ \frac{dr}{dt} \right)$ and $\phi_2 = \tan^{-1}\left(r \Big/ \frac{dr}{dt} \right)$

Step 5: So, $|\phi_1 - \phi_2|$ gives angle between the given curves.

Radius of curvature:

1. Formula to find radius of curvature ρ in Cartesian form is $\rho = \frac{\left(1 + y_1^2\right)^{3/2}}{y_2}$

where $y_1 = \frac{dy}{dx}$ & $y_2 = \frac{d^2y}{dx^2} = \frac{dy_1}{dx}$

2. Formula to find radius of curvature ρ in polar form is $\rho = \frac{\left(r^2 + r_1^2\right)^{3/2}}{r^2 + 2r_1^2 - r\,r_2}$

where $r_1 = \frac{dr}{d\theta}$ & $r_2 = \frac{d^2r}{d\theta^2} = \frac{dr_1}{d\theta}$

3. Formula to find radius of curvature ρ in parametric form is $\rho = \frac{\left(x_1^2 + y_1^2\right)^{3/2}}{x_2 y_1 - y_2 x_1}$

where $x_1 = \frac{dx}{dt}$, $x_2 = \frac{d^2x}{dt^2}$, $y_1 = \frac{dy}{dt}$ & $y_2 = \frac{d^2y}{dt^2}$

Python modules used in the lab:

```
diff(func, symbols)   # returns derivative of func with respect to symbols
solve(f, symbols) # algebraically solves equations and system of equations.
                   Supports polynomial, transcendental, systems of linear
                   and polynomial equations and system containing relational
                   expressions
simplify(expr) # simplifies the given expression
ratsimp(expr)  # Put an expression over a common denominator, cancel and
                  reduce.
expr.subs({x:2,y:4}) # replaces each occurrence of x by 2 and y by 4 in
                       expr
display(objects)  #Display a Python object in all frontends

Note to use special characters like ρ(rho), type \rho and press tab
```

Program 1: Program to find the angle between two polar curves $r = e^{at}$ and $r = e^{-at}$

```
1  from sympy import *
2  a,t,φ1,φ2=symbols('a,t,φ1,φ2')
3  r1=exp(a*t)
4  r2=exp(-a*t)
5  θ=solve(r1-r2,t)  #to find point of intersection
6  print("The point of intersection is",θ)
7  dr1=diff(r1,t)
8  r1dr1=r1/dr1   #tan(ϕ1)
9  dr2=diff(r2,t)
10 r2dr2=r2/dr2   #tan(ϕ2)
11 r1dr1=r1dr1.subs({t:θ[0]})    #tan(ϕ1) at ϑ
12 r2dr2=r2dr2.subs({t:θ[0]})    #tan(ϕ2) at ϑ
13 p1=atan(r1dr1)
14 p2=atan(r2dr2)
15 p=abs(p1-p2)
16 print("Angle between the curves is ")
17 display(Eq(abs(φ1-φ2),p))
18 print("or in degrees")
19 display(Eq(abs(φ1-φ2),deg(p)))
```

Output:

```
The point of intersection is [0, I*pi/a]
Angle between the curves is
```

$$|\phi_1 - \phi_2| = 2\left|\text{atan}\left(\frac{1}{a}\right)\right|$$

```
or in degrees
```

$$|\phi_1 - \phi_2| = \frac{360\left|\text{atan}\left(\frac{1}{a}\right)\right|}{\pi}$$

Program 2: Program to find the angle between two polar curves $r = 4(1+\cos(t))$ and $r = 5(1-\cos(t))$

```
from sympy import *
t,φ1,φ2=symbols('t,φ1,φ2')
r1=4*(1+cos(t))
r2=5*(1-cos(t))
θ=solve(r1-r2,t)  #to find point of intersection
print("The point of intersection is",θ)
dr1=diff(r1,t)
r1dr1=r1/dr1   #tan(ϕ1)
dr2=diff(r2,t)
r2dr2=r2/dr2   #tan(ϕ2)
r1dr1=r1dr1.subs({t:θ[0]})    #tan(ϕ1) at ϑ
r2dr2=r2dr2.subs({t:θ[0]})    #tan(ϕ2) at ϑ
p1=atan(r1dr1)
p2=atan(r2dr2)
p=abs(p1-p2)
print("Angle between the curves is ")
display(Eq(abs(φ1-φ2),float(p)))
print("or in degrees")
display(Eq(abs(φ1-φ2),float(deg(p))))
```

Output:

```
The point of intersection is [-acos(1/9) + 2*pi, acos(1/9)]
Angle between the curves is
```

$$|\phi_1 - \phi_2| = 1.5707963267949$$

```
or in degrees
```

$$|\phi_1 - \phi_2| = 90.0$$

Program 3: program to find the radius of curvature and curvature for the curve (polar form) $r = a\sin(nt)$ at the point $t = \pi/2$ and $n = 1$

```
from sympy import *
a,n,t,ρ=symbols("a,n,t,ρ")
r=a*sin(n*t)
r1=diff(r,t)
r2=diff(r1,t)
rho=(r**2+r1**2)**(3/2)/(r**2+2*r1**2-r*r2)
print("Radius of curvatre is")
display(Eq(ρ,rho))
rho=rho.subs({t:pi/2,n:1})
print("Radius of curvatre at t=pi/2 and n=1 is")
display(Eq(ρ,rho))
print("Curvatre at t=pi/2 and n=1 is ")
display(Eq(1/ρ,1/rho))
```

Output:

```
Radius of curvatre is
```

$$\rho = \frac{\left(a^2 n^2 \cos^2(nt) + a^2 \sin^2(nt)\right)^{1.5}}{a^2 n^2 \sin^2(nt) + 2a^2 n^2 \cos^2(nt) + a^2 \sin^2(nt)}$$

```
Radius of curvatre at t=pi/2 and n=1 is
```

$$\rho = \frac{\left(a^2\right)^{1.5}}{2a^2}$$

```
Curvatre at t=pi/2 and n=1 is
```

$$\frac{1}{\rho} = \frac{2a^2}{\left(a^2\right)^{1.5}}$$

Program 4: Program to find the radius of curvature and curvature for the curve (parametric form) $x = a\cos(t)$ and $y = a\sin(t)$ with $a = 5$ at $t = \pi/2$

```
from sympy import *
t,a,ρ=symbols("t,a,ρ") # define all symbols required
x=a*cos(t)
x1=diff(x,t)
x2=diff(x1,t)
y=a*sin(t)
y1=diff(y,t)
y2=diff(y1,t)
rho=(x1**2+y1**2)**(3/2)/(y1*x2-x1*y2)
print('Radius of curvature is ')
display(Eq(ρ,ratsimp(rho)))
rho=rho.subs({t:pi/2,a:5})
print('\nRadius of curvature at a=5 and t = pi/2 is')
display(Eq(ρ,rho))
print ('\n Curvature at a=5 and t = pi/2 is ')
display(Eq(1/ρ,1/rho))
```

Output:

```
Radius of curvature is
```

$$\rho = -\left(a^2 \sin^2(t) + a^2 \cos^2(t)\right)^{0.5}$$

```
Radius of curvature at a=5 and t = pi/2 is
```

$$\rho = -5.0$$

```
Curvature at a=5 and t = pi/2 is
```

$$\frac{1}{\rho} = -0.2$$

Exercise: Write python program for the following

1. Find the angle between two polar curves
   a) $r = 4\cos(t)$ and $r = 5\sin(t)$
   b) $r = at$ and $r = a/t$
   c) $r = 2(1+\sin(t))$ and $r = 2(1+\cos(t))$
   d) $r = 2\cos(t)$ and $r = 2\sin(t)$
2. Find radius of curvature and curvature of the curve
   a) $r = 2(1-\cos(t))$ at $t = \frac{\pi}{2}$
   b) $r = a\cos(t)$ at $t = \frac{\pi}{4}$
3. Find radius of curvature and curvature of the curve
   a) $x = a\cos^3(t), y = a\sin^3(t)$ at $t = \frac{\pi}{4}$
   b) $x = a(t-\sin(t)), y = a(1-\cos(t))$ at $t = \frac{\pi}{2}$
   c) $x = a(3\cos t - \cos(3t)), y = a(3\sin t - \sin(3t))$ at $t = \frac{\pi}{2}$

OBSERVATIONS

LAB 3: Finding Partial derivatives and Jacobian of functions of several variables

Partial differentiation:

If $u = f(x_1, x_2, x_3, ..., x_n)$ then differentiation of u with respect to x_i by keeping remaining n-1 variables as constants is called partial differentiation u with respect to x_i.

The partial derivative of u with respect to x_i is denoted as $\frac{\partial u}{\partial x_i}$ or $\frac{\partial f}{\partial x_i}$.

Example:

If $u = f(x, y) = x^2 + y^3 + e^{xy} + \sin(3x + 2y)$ then

$$u_x = \frac{\partial u}{\partial x} = 2x + ye^{xy} + 3\cos(3x + 2y) \qquad u_y = \frac{\partial u}{\partial y} = 3y^2 + xe^{xy} + 2\cos(3x + 2y)$$

$$u_{xx} = \frac{\partial^2 u}{\partial x^2} = \frac{\partial}{\partial x}\left(\frac{\partial u}{\partial x}\right) = 2 + y^2 e^{xy} - 9\sin(3x + 2y)$$

$$u_{yy} = \frac{\partial^2 u}{\partial y^2} = \frac{\partial}{\partial y}\left(\frac{\partial u}{\partial y}\right) = 6y + x^2 e^{xy} - 4\sin(3x + 2y)$$

$$u_{xy} = \frac{\partial^2 u}{\partial x \partial y} = \frac{\partial}{\partial x}\left(\frac{\partial u}{\partial y}\right) = \frac{\partial}{\partial y}\left(\frac{\partial u}{\partial x}\right) = xye^{xy} - 6\sin(3x + 2y)$$

Jacobian:

If $u = f(x, y)$ and $v = g(x, y)$ then the Jacobian of u & v with respect to x & y is

$$J = \frac{\partial(u, v)}{\partial(x, y)} = \begin{vmatrix} \frac{\partial u}{\partial x} & \frac{\partial u}{\partial y} \\ \frac{\partial v}{\partial x} & \frac{\partial v}{\partial y} \end{vmatrix}$$

Similarly, If $u = f(x, y, z), v = g(x, y, z)$ and $w = h(x, y, z)$ then the Jacobian of u, v, w with respect to x, y & z is $J = \frac{\partial(u, v, w)}{\partial(x, y, z)} = \begin{vmatrix} \frac{\partial u}{\partial x} & \frac{\partial u}{\partial y} & \frac{\partial u}{\partial z} \\ \frac{\partial v}{\partial x} & \frac{\partial v}{\partial y} & \frac{\partial v}{\partial z} \\ \frac{\partial w}{\partial x} & \frac{\partial w}{\partial y} & \frac{\partial w}{\partial z} \end{vmatrix}$

Example:

Find the Jacobian, if $u = x + y$ & $v = xy$.

Soln: Given $u = x + y$ & $v = xy$

$$\frac{\partial u}{\partial x}=1,\ \frac{\partial u}{\partial y}=1,\ \frac{\partial v}{\partial x}=y\ \&\frac{\partial v}{\partial y}=x$$

So, $J=\frac{\partial(u,v)}{\partial(x,y)}=\begin{vmatrix}1 & 1\\ y & x\end{vmatrix}=x-y$

Python modules used in the lab:

```
f=Function("u")(x,y)    # f represnts function u which depends on variables
                          x and y
Derivative(expr, reference variables) #returns an unevaluated derivative of
                                  expr with respect to reference variables
Derivative(expr, reference variables, evaluate=True) #returns derivative of
                                  expr with respect to reference variables
Matrix([[row1], [row2], [row3],… [rown]])  #creates the matrix of n rows
Matrix([[row1], [row2], [row3]])  #creates the matrix of 3 rows
Determinant(mat)   # Represents the determinant of a given matrix mat
det(mat)    # returns the determinant value of the given matrix mat
```

Program 1: Program to find first and second order partial derivatives of $u=f(x,y)=e^x(x\ \cos y\ -\ y\ \sin y)$

```
1  from sympy import *
2  x,y=symbols("x,y")
3  f=Function("u")(x,y)
4  u=exp(x)*(x*cos(y)-y*sin(y))
5
6  display(Eq(Derivative(f,x),diff(u,x)))
7  display(Eq(Derivative(f,y),diff(u,y)))
8  display(Eq(Derivative(f,x,2),diff(u,x,2)))
9  display(Eq(Derivative(f,y,2),diff(u,y,2)))
10 display(Eq(Derivative(f,y,x),diff(u,y,x)))
```

Output

$$\frac{\partial}{\partial x}u(x,y)=(x\cos(y)-y\sin(y))\,e^x+e^x\cos(y)$$

$$\frac{\partial}{\partial y}u(x,y)=(-x\sin(y)-y\cos(y)-\sin(y))\,e^x$$

$$\frac{\partial^2}{\partial x^2}u(x,y)=(x\cos(y)-y\sin(y)+2\cos(y))\,e^x$$

$$\frac{\partial^2}{\partial y^2}u(x,y)=-(x\cos(y)-y\sin(y)+2\cos(y))\,e^x$$

$$\frac{\partial^2}{\partial x\partial y}u(x,y)=-(x\sin(y)+y\cos(y)+2\sin(y))\,e^x$$

Program 2: Program to show $\frac{\partial u}{\partial x}+\frac{\partial u}{\partial y}+\frac{\partial u}{\partial z}=\frac{3}{x+y+z}$ if $u=\log(x^3+y^3+z^3-3xyz)$

```
from sympy import *
x,y,z=symbols("x,y,z")
f=Function("u")(x,y,z)
u=log(x**3+y**3+z**3-3*x*y*z)

ux=simplify(diff(u,x))
display(Eq(Derivative(f,x),ux))
uy=simplify(diff(u,y))
display(Eq(Derivative(f,y),uy))
uz=simplify(diff(u,z))
display(Eq(Derivative(f,z),uz))
LHS=Derivative(f,x)+Derivative(f,y)+Derivative(f,z)
RHS=simplify(ux+uy+uz)
display(Eq(LHS,RHS))
```

Output:

$$\frac{\partial}{\partial x}u(x,y,z)=\frac{3\left(x^2-yz\right)}{x^3-3xyz+y^3+z^3}$$

$$\frac{\partial}{\partial y}u(x,y,z)=\frac{3\left(-xz+y^2\right)}{x^3-3xyz+y^3+z^3}$$

$$\frac{\partial}{\partial z}u(x,y,z)=\frac{3\left(-xy+z^2\right)}{x^3-3xyz+y^3+z^3}$$

$$\frac{\partial}{\partial x}u(x,y,z)+\frac{\partial}{\partial y}u(x,y,z)+\frac{\partial}{\partial z}u(x,y,z)=\frac{3}{x+y+z}$$

Program 3: Program to show $a^2\frac{\partial^2 u}{\partial x^2}-\frac{\partial u}{\partial t}=0$ if $u=At^{-\frac{1}{2}}e^{-\frac{x^2}{4a^2t}}$

```
from sympy import *
A, a, t, x = symbols("A,a,t,x")
f=Function("u")(x,t)
u=A*t**(-1/2)*exp(-x**2/(4*a**2*t))

uxx=simplify(diff(u,x,2))
display(Eq(Derivative(f,x,2),uxx))
ut=simplify(diff(u,t))
display(Eq(Derivative(f,t),ut))
LHS=a**2*Derivative(f,x,2)-Derivative(f,t)
RHS=simplify(a**2*uxx-ut)
display(Eq(LHS,RHS))
```

Output:

$$\frac{\partial^2}{\partial x^2}u(x,t) = \frac{A\left(-2a^2t + x^2\right)e^{-\frac{x^2}{4a^2t}}}{4a^4t^{2.5}}$$

$$\frac{\partial}{\partial t}u(x,t) = \frac{A\left(-2.0a^2t^{2.5} + t^{1.5}x^2\right)e^{-\frac{x^2}{4a^2t}}}{4a^2t^{4.0}}$$

$$a^2\frac{\partial^2}{\partial x^2}u(x,t) - \frac{\partial}{\partial t}u(x,t) = 0$$

Program 4: Program to find the Jacobian, if $u = \frac{xy}{z}, v = \frac{yz}{x}, w = \frac{zx}{y}$

```
from sympy import *
x,y,z=symbols("x,y,z")
U=x*y/z
V=y*z/x
W=z*x/y
Ux=simplify(diff(U,x))
Uy=simplify(diff(U,y))
Uz=simplify(diff(U,z))
Vx=simplify(diff(V,x))
Vy=simplify(diff(V,y))
Vz=simplify(diff(V,z))
Wx=simplify(diff(W,x))
Wy=simplify(diff(W,y))
Wz=simplify(diff(W,z))
M=Matrix([[Ux,Uy,Uz],[Vx,Vy,Vz],[Wx,Wy,Wz]])
J=simplify(det(M))
display(Eq(Determinant(M),J))
```

Output:

$$\begin{vmatrix} \frac{y}{z} & \frac{x}{z} & -\frac{xy}{z^2} \\ -\frac{yz}{x^2} & \frac{z}{x} & \frac{y}{x} \\ \frac{z}{y} & -\frac{xz}{y^2} & \frac{x}{y} \end{vmatrix} = 4$$

Exercise: Write python program for the following

1. Show $\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$ if $u = e^x(x\ \cos y\ -\ y\ \sin y)$

2. Verify $\frac{\partial^2 u}{\partial x \partial y} = \frac{\partial^2 u}{\partial y \partial x}$ if $u = \tan^{-1}\left(\frac{y}{x}\right)$ (Hint: use $a\tan(x)$ for $\tan^{-1}(x)$)

3. Show $x\frac{\partial u}{\partial x}+y\frac{\partial u}{\partial y}=1$ if $u=\log\left(\frac{x^2+y^2}{x+y}\right)$

4. If $x=r\cos(t)$ & $y=r\sin(t)$, find $\frac{\partial(x,y)}{\partial(r,t)}$

5. If $u=\rho\cos(\varphi)\sin(\theta), v=\rho\cos(\varphi)\cos(\theta), w=\rho\sin(\varphi)$, find $J=\frac{\partial(u,v,w)}{\partial(\rho,\varphi,\theta)}$

6. If $u=x+3y^2-z^3, v=4x^2yz, w=2z^2-xy$, find $J=\frac{\partial(u,v,w)}{\partial(x,y,z)}$ at $(-2,-1,1)$

OBSERVATIONS

LAB 4: Finding Maclaurin's series expansion, Limit of a function and Maxima and Minima of functions of two variables

Maclaurin's series expansion for a function of one variable:

The Maclaurin's series expansion of $y = f(x)$ is

$$y(x) = y(0) + xy_1(0) + \frac{x^2}{2!}y_2(0) + \frac{x^3}{3!}y_3(0) + \cdots$$

where $y_1 = f'(x), y_2 = f''(x), y_3 = f'''(x)...$

Example:

Find Maclaurin's series of $y = e^x$ up to fourth degree term.

Soln:

$$y = e^x \quad \Rightarrow y(0) = 1$$

$$y_1 = e^x \quad \Rightarrow y_1(0) = 1$$

$$y_2 = e^x \quad \Rightarrow y_2(0) = 1$$

$$y_3 = e^x \quad \Rightarrow y_3(0) = 1$$

$$y_4 = e^x \quad \Rightarrow y_4(0) = 1$$

We have

$$y(x) = y(0) + xy_1(0) + \frac{x^2}{2!}y_2(0) + \frac{x^3}{3!}y_3(0) + \frac{x^4}{4!}y_4(0) + \cdots$$

Hence, $y(x) = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots$

Indeterminate forms:

In the process of evaluating certain limits, sometimes we get an expression of the form $\frac{0}{0}, \frac{\infty}{\infty}, 0 \times \infty, \infty - \infty, 0^0, \infty^0$ which do not represent any real value. The expressions of this kind are called Indeterminate forms. We use L'Hospitals rule to evaluate such limits. In python we have inbuilt function for the same.

Maxima and Minima of function $f(x, y)$:

The extremum point (a, b) is a (relative) maximum point of $f(x, y)$ whenever the value of $f(x, y)$ at the point (a, b) is greater than the value of $f(x, y)$ at a neighbouring point. Then

$f(a,b)$ is called a (relative) maximum value or a maxima. Next, we say that the extremum point (a,b) is a (relative) minimum point of $f(x,y)$ whenever the value of $f(x,y)$ at the point (a,b) is less than the value of $f(x,y)$ at a neighbouring point. Then $f(a,b)$ is called a (relative) minimum value or a minima.

The working rule for the determination of maxima and minima of $f(x,y)$:

Step 1: Find partial derivatives $p=\frac{\partial f}{\partial x}, q=\frac{\partial f}{\partial y}, r=\frac{\partial^2 f}{\partial x^2}, s=\frac{\partial^2 f}{\partial x \partial y}$ & $t=\frac{\partial^2 f}{\partial y^2}$

Step 2: Each pair of values (a,b) of (x,y) obtained by solving $p=0$ & $q=0$ determines an extremum point of $f(x,y)$

Step 3: For each extremum point (a,b), find $A=(r)_{(a,b)}$; $B=(s)_{(a,b)}$; $C=(t)_{(a,b)}$ and $\delta = AC-B^2$.

Step 4: At an extremum point (a,b)

Case a: If $A<0$ and $\delta>0$, then (a,b) is maximum point and $f(a,b)$ is maxima

Case b: If $A>0$ and $\delta>0$, then (a,b) is minimum point and $f(a,b)$ is minima

Case c: If $\delta<0$, then (a,b) is called saddle point

Case d: If $A=0$ and/or $\delta=0$, further analysis is necessary to decide the nature

Example:

Find the extreme values of $f(x,y)=x^4+y^4-2x^2-2y^2+4xy$

Soln:

$p=\frac{\partial f}{\partial x}=4x^3-4x+4y$ $\qquad$ $q=\frac{\partial f}{\partial y}=4y^3+4x-4y$

$r=\frac{\partial^2 f}{\partial x^2}=12x^2-4$ $\qquad$ $s=\frac{\partial^2 f}{\partial x \partial y}=4$ $\qquad$ $t=\frac{\partial^2 f}{\partial y^2}=12y^2-4$

The stationary points on solving $p=0$ & $q=0$ are $(0,0)$, $(-\sqrt{2},\sqrt{2})$ and $(\sqrt{2},-\sqrt{2})$.

Since $\delta = AC-B^2=0$ at $(0,0)$, further analysis is necessary to decide the nature.

At $(-\sqrt{2},\sqrt{2})$, $\delta = AC-B^2=384>0$ and $A=20>0$. So $(-\sqrt{2},\sqrt{2})$ is minimum point and the minimum value is $f(-\sqrt{2},\sqrt{2})=-8$.

At $\left(\sqrt{2},-\sqrt{2}\right)$, $\delta = AC - B^2 = 384 > 0$ and $A = 20 > 0$. So $\left(\sqrt{2},-\sqrt{2}\right)$ is minimum point and the minimum value is $f\left(\sqrt{2},-\sqrt{2}\right) = -8$.

Python modules used in the lab:

```
f=lambdify(x, exp) # Convert a SymPy expression into a function f(x) that
            allows for fast numeric evaluation. Lambdify acts like a lambda
             function, except it, converts Sympy names to the names of the
            given numerical library, usually Numpy or math
type(object)   #returns object's type
Limit(e, z, z0)  #Represents an unevaluated limit of e as z tends to z0
Limit(e, z, z0)  # Computes the limit of ``e(z)`` at the point ``z0``
```

Program 1: Program to find the Maclaurin's series expansion of $y = \log(1+x)$

```
1 from sympy import *
2 x=symbols("x")
3 y=log(1+exp(x))
4 ys=0
5 n=int(input("Enter the degree of polynomial required: "))
6 for i in range(0,n+1):
7     ys+=x**i*diff(y,x,i).subs({x:0})/factorial(i)
8 print("Maclaurin's series is")
9 display(Eq(y,ys))
```

Output:

```
Enter the degree of polynomial required: 6
Maclaurin's series is
```

$$\log(e^x + 1) = \frac{x^6}{2880} - \frac{x^4}{192} + \frac{x^2}{8} + \frac{x}{2} + \log(2)$$

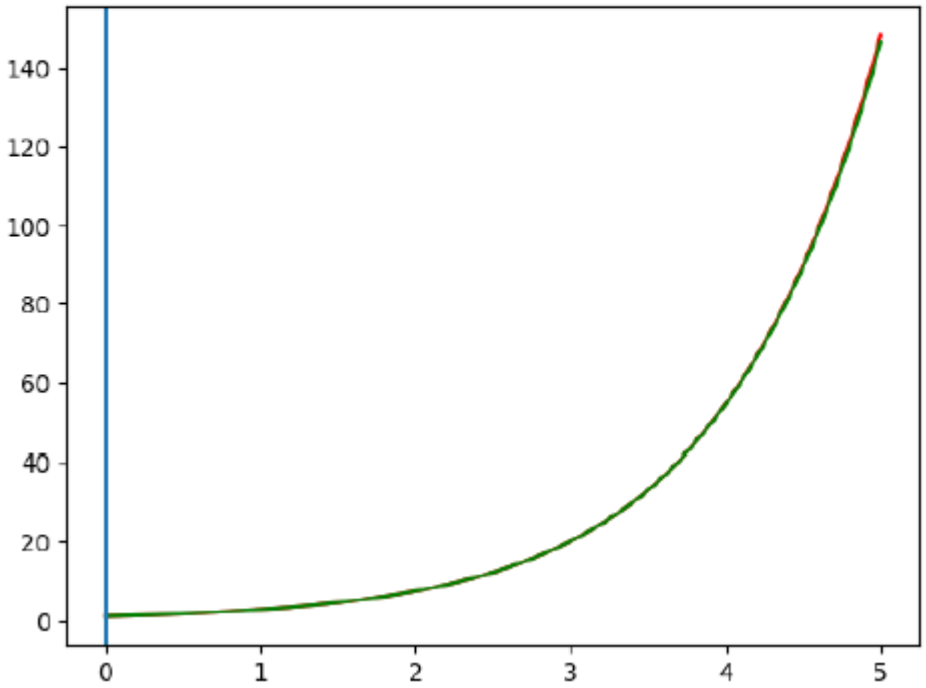
Program 2: Program to find the Maclaurin's series expansion of $y = e^x$ and plot it over the interval $(0,5)$.

```
1  from sympy import *
2  x=symbols("x")
3  y=exp(x)
4  ys=0
5  n=int(input("Enter the degree of polynomial required: "))
6  for i in range(0,n+1):
7      ys+=x**i*diff(y,x,i).subs({x:0})/factorial(i)
8  print("Maclaurin's series is")
9  display(Eq(y,ys))
10 ys=lambdify(x,ys)
11 from pylab import *
12 x=linspace(0,5,1000)
13 plot(x,exp(x),color="red")
14 plot(x,ys(x),color="green")
15 axvline()
16 show()
```

Output:

```
Enter the degree of polynomial required: 10
Maclaurin's series is
```

$$e^x = \frac{x^{10}}{3628800} + \frac{x^9}{362880} + \frac{x^8}{40320} + \frac{x^7}{5040} + \frac{x^6}{720} + \frac{x^5}{120} + \frac{x^4}{24} + \frac{x^3}{6} + \frac{x^2}{2} + x + 1$$



Program 3: Program to evaluate $\lim_{x\to 0}\left(\frac{a^x - b^x}{x}\right)$

```
from sympy import *
a,b,x=symbols("a,b,x")
f=(a**x-b**x)/x
LHS=Limit(f,x,0)
RHS=limit(f,x,0)
display(Eq(LHS,RHS))
```

Output:

$$\lim_{x\to 0^+}\left(\frac{a^x - b^x}{x}\right) = \log(a) - \log(b)$$

Program 4: Program to evaluate $\lim_{x\to\infty}\left(1+\frac{1}{x}\right)^x$

```
from sympy import *
x=symbols("x")
f=(1+1/x)**x
LHS=Limit(f,x,oo)
RHS=limit(f,x,oo)
display(Eq(LHS,RHS))
```

Output:

$$\lim_{x\to\infty}\left(1+\frac{1}{x}\right)^x = e$$

Program 5: Program to find extreme values of $f(x,y)=x^4+y^4-2x^2-2y^2+4xy$

```
from sympy import *
x,y=symbols('x,y')
f=(x**4)+(y**4)-(2*x**2)+(4*x*y)-(2*y**2)
#Finding derivatives
fx = diff(f,x)
fy = diff(f,y)
fxx = diff(fx,x)
fxy = diff(fx,y)
fyy = diff(fy,y)
#Finding extremum points
points=solve([Eq(fx,0),Eq(fy,0)],[x,y])
a,b=[],[]
if type(points)!=list:
    a.append(points[x])
    b.append(points[y])
else:
    for i in range(len(points)):
        (a1,b1)=points[i]
        if im(a1)==0 and im(b1)==0:
            a.append(a1)
            b.append(b1)
print("The extremum points are:")
for i in range(len(a)):
    print(f"{a[i],b[i]}",end=" ")
print()
# Determining nature of extremum points
for i in range(len(a)):
    A=fxx.subs({x:a[i],y:b[i]})
    B=fxy.subs({x:a[i],y:b[i]})
    C=fyy.subs({x:a[i],y:b[i]})
    delta=A*C-B**2
    print('\nA=',A,'\tB=',B,'\tC=',C,"\tAC-B^2=",delta)
    if delta>0 and A>0:
        print('(%0.2f,%0.2f)'%(a[i],b[i]),'is the minimum point')
        print("f(%0.2f,%0.2f)=%.4f"%(a[i],b[i],f.subs({x:a[i],y:b[i]})))
    elif delta>0 and A<0:
        print('(%0.2f,%0.2f)'%(a[i],b[i]),'is the maximum point')
        print("f(%0.2f,%0.2f)=%.4f"%(a[i],b[i],f.subs({x:a[i],y:b[i]})))
    elif delta<0:
        print('(%0.2f,%0.2f)'%(a[i],b[i]),'is the saddle point')
    elif delta==0:
        print("At (%0.2f,%0.2f)"%(a[i],b[i]),' Further investigation is required ')
```

Output:

```
The extremum points are:
(0, 0) (-sqrt(2), sqrt(2)) (sqrt(2), -sqrt(2))

A= -4   B= 4    C= -4   AC-B^2= 0
At (0.00,0.00)  Further investigation is required

A= 20   B= 4    C= 20   AC-B^2= 384
(-1.41,1.41) is the minimum point
f(-1.41,1.41)=-8.0000

A= 20   B= 4    C= 20   AC-B^2= 384
(1.41,-1.41) is the minimum point
f(1.41,-1.41)=-8.0000
```

Exercise: Write python program for the following

1. Find the Maclaurin's series expansion of the following
   a) $y=\log(1+x)$ b) $y=\sqrt{1+\sin 2x}$ c) $y=\tan^{-1}x$
2. Find the Maclaurin's series of $y=e^{-2x}$ and plot it over the interval $(0,5)$
3. Find the Maclaurin's series of $y=\cos(x)$ and plot it over the interval $(0,2\pi)$
4. Find the Maclaurin's series of $y=\cos(x)+\sin(x)$ and plot it over the interval $(0,2\pi)$
5. Find the Maclaurin's series of $y=\tan(x)$ and plot it over the interval $(0,\pi)$
6. Evaluate the following

   a) $\lim\limits_{x\to\pi/2}\dfrac{\log\sin x}{(\pi/2-x)^2}$ b) $\lim\limits_{x\to 0}\dfrac{1-\cos x}{x\log(1+x)}$ c) $\lim\limits_{x\to 0}\dfrac{x-\tan x}{x^3}$

   d) $\lim\limits_{x\to\pi/4}(\tan x)^{\tan 2x}$ e) $\lim\limits_{x\to 0}\left(\dfrac{a^x+b^x+c^x}{3}\right)^{1/x}$ f) $\lim\limits_{x\to\infty}\left(\dfrac{ax+1}{ax-1}\right)^x$

   g) $\lim\limits_{x\to\infty}\left(\dfrac{1^{1/x}+2^{1/x}+3^{1/x}}{3}\right)^{3x}$ h) $\lim\limits_{x\to 0}\left(\dfrac{\tan x}{x}\right)^{1/x^2}$

7. Find extreme values of the following:
   a) $f(x,y)=x^2+y^2+6x-12$ b) $f(x,y)=x^3+y^3-3x-12y+20$
   c) $f(x,y)=x^3y^2(1-x-y)$ d) $f(x,y)=x^2+xy+y^2+\dfrac{1}{x}+\dfrac{1}{y}$

OBSERVATIONS

LAB 5: Solving first-order ODEs, plotting solutions of IVPs and Application Problems Ordinary differential equations of First-order and First-degree:

If x is independent variable and y is the dependent variable then the standard form first-order and first-degree ordinary differential equation is

$$f\left(x, y, \frac{dy}{dx}\right) = 0$$

in which $\frac{dy}{dx}$ appears with first-degree. The general solution of this equation will be of the form $F(x, y, c) = 0$, where c is an arbitrary constant.

Further, if y is known at $x = 0$, then the above differential equation with this condition specified is called initial value problem(IVP) and the solution obtained is called particular solution.

The procedure to solve the above differential equation depends on the nature of the function f.

Here, we will use the python module dsolve( ) for the above purpose.

Python modules used in the lab:

```
dsolve(eq, f(x), hint, ics) # Solves any (supported) kind of ordinary
                              differential equation and system of ordinary
                              differential equations

where,

``eq`` can be any supported ordinary differential equation. This can either
be an Equality, or an expression, which is assumed to be equal to ``0``.

``f(x)`` is a function of one variable whose derivatives in that variable

make up the ordinary differential equation ``eq``. In many cases it is not
necessary to provide this; it will be auto-detected (and an error raised if
it couldn't be detected)

``hint`` is the solving method that you want dsolve to use. Use
``classify_ode(eq, f(x))`` to get all of the possible hints for an ODE.

The default hint, ``default``, will use whatever hint is returned first.

``ics`` is the set of initial/boundary conditions for the differential equa
tion.
```

Program 1: Program to solve $\frac{dy}{dx}+y\tan(x)=y^3\sec(x)$

```
from sympy import *
x=Symbol("x")
y=Function("y")(x)
DE=Eq(Derivative(y,x)+y*tan(x)-y**3*sec(x),0)
display(DE)
GS=dsolve(DE,y)
if type(GS)==Equality:
    display(GS)
else:
    for i in GS:
        display(i)
```

Output:

$$-y^3(x)\sec(x)+y(x)\tan(x)+\frac{d}{dx}y(x)=0$$

$$y(x)=-\sqrt{\frac{1}{C_1-2\sin(x)}}\cos(x)$$

$$y(x)=\sqrt{\frac{1}{C_1-2\sin(x)}}\cos(x)$$

Program 2: Program to solve initial value problem $\frac{dP}{dt}=r$ with $P(0)=2$, take $r=3$. Also plot the solution.

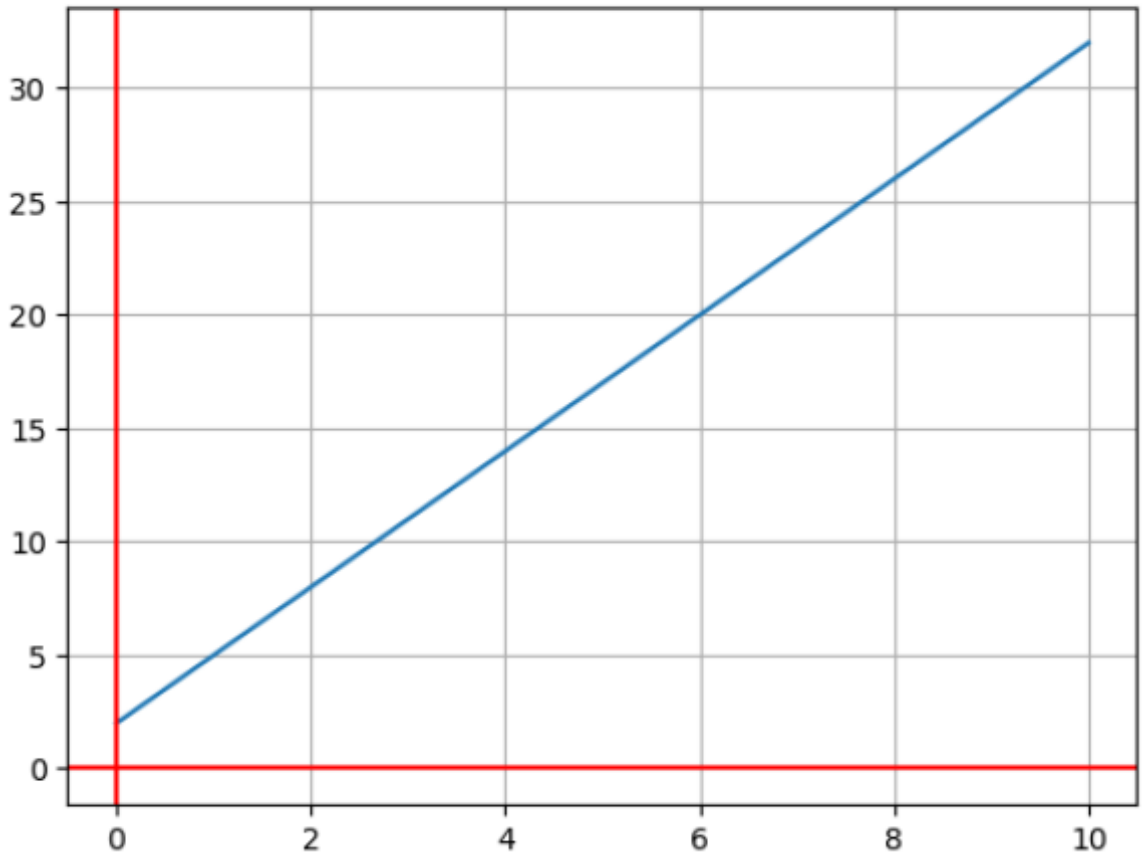
```
from sympy import *
r,t=symbols("r,t")
P=Function("P")(t)
DE=Eq(Derivative(P,t),r)
display(DE)
PS=dsolve(DE,P,ics={P.subs({t:0}):2}).subs({r:3})
print("Solution of IVP is: ")
display(PS)
PS=lambdify(t,PS.rhs)
from pylab import *
t=linspace(0,10,100)
plot(t,PS(t))
axvline(color="red")
axhline(color="red")
grid()
show()
```

Output:

$$\frac{d}{dt}P(t) = r$$

Solution of IVP is:

$$P(t) = 3t + 2$$



Program 3:

Newton's law of cooling
The temperature of a body drops from 100 ºC to 80 ºC in 10 minutes where the surrounding air is at the temperature 25 ºC. Write a program to solve this initial value problem and plot the graph of cooling.

The mathematical model (IVP) for the above situation is:

$\frac{d\theta}{dt} = -k(\theta - \theta_m)$, $\theta(0) = 100$ and $\theta(10) = 80$.

where the temperature (θ) is time (t) dependent and $\theta_m = 25$

```
from sympy import *
t=Symbol("t")
k=Symbol("k",real=True)
θ=Function("θ")(t)
DE=Eq(Derivative(θ,t)+k*(θ-25),0)
display(DE)
PS=dsolve(DE,θ,ics={θ.subs({t:0}):100})
print("Solution of IVP is ")
display(PS)
sol_k=solve(PS.subs({θ:80,t:10}),k)
PS=PS.subs({k:sol_k[0]})
print("Hence the solution is: ")
display(PS)
PS=lambdify(t,PS.rhs)
from pylab import *
t=arange(0,100,0.01)
plot(t,PS(t))
axvline(color="red")
axhline(color="red")
grid()
show()
```

Output:

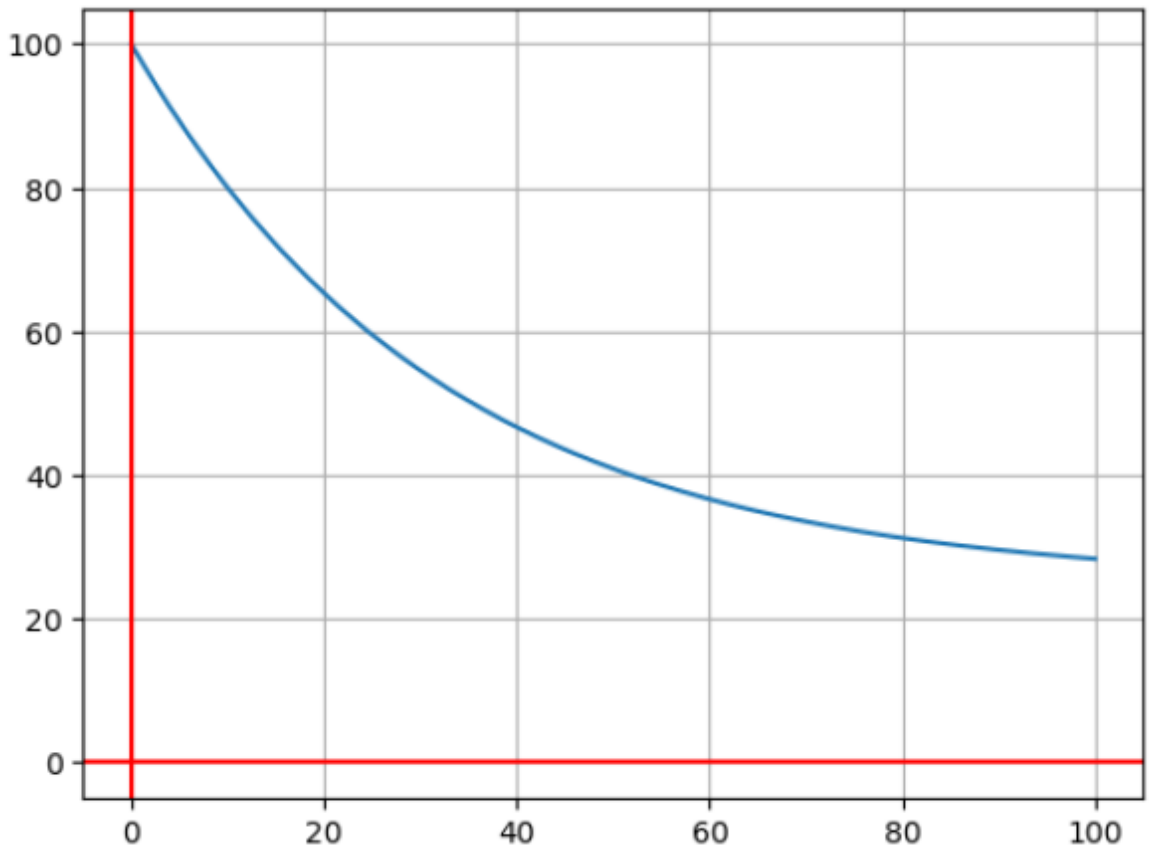
$$k\,(\theta(t) - 25) + \frac{d}{dt}\theta(t) = 0$$

Solution of IVP is

$$\theta(t) = 25 + 75e^{-kt}$$

Hence the solution is:

$$\theta(t) = 25 + 75e^{-t\log\left(\frac{11^{\frac{9}{10}}\cdot\sqrt[10]{15}}{11}\right)}$$



Program 4: Consider the following two situations representing same mathematical model:

Growth of Virus affected files in a system

If number of virus affected files in a system is proportional to the number of virus affected files $P(t)$ at time t, then determine the number of virus affected files at time t. Assume that the number virus affected files in the system is P_0 to begin with and after $t=1$hour the number virus affected files found to be $(3/2)P_0$. Take $P_0 = 20$.

OR

Growth of Bacteria in a culture test

If the rate of growth of bacteria is proportional to the number of bacteria $P(t)$ present at time t, determine the number of bacteria at time t. Assume the culture initially has P_0 number of bacteria and after $t=1$ hour the number of bacteria measured to be $(3/2)P_0$. Take $P_0 = 20$.

The mathematical model for the above situation is

$\frac{dP}{dt} = kP$, $P(0) = P_0$ and $P(1) = (3/2)P_0$. Assume $P_0 = 20$.

Write a program to solve this IVP.

```
from sympy import *
t,k,P0=symbols("t,k,P0")
P=Function("P")(t)
DE=Eq(Derivative(P,t)-k*P,0)
print("Differential Equation is: ")
display(DE)
PS=dsolve(DE,P,ics={P.subs({t:0}):P0})
print("Particular Solution is: ")
display(PS)
sol_k=solve(PS.subs({P:(3/2)*P0,t:1}),k)
PS=PS.subs({k:sol_k[0],P0:20})
print("Hence Solution is: ")
display(PS)
PS=lambdify(t,PS.rhs)
from pylab import *
t=linspace(0,10,100)
plot(t,PS(t))
axvline(color="red")
axhline(color="red")
grid()
show()
```

Output:

Differential Equation is:

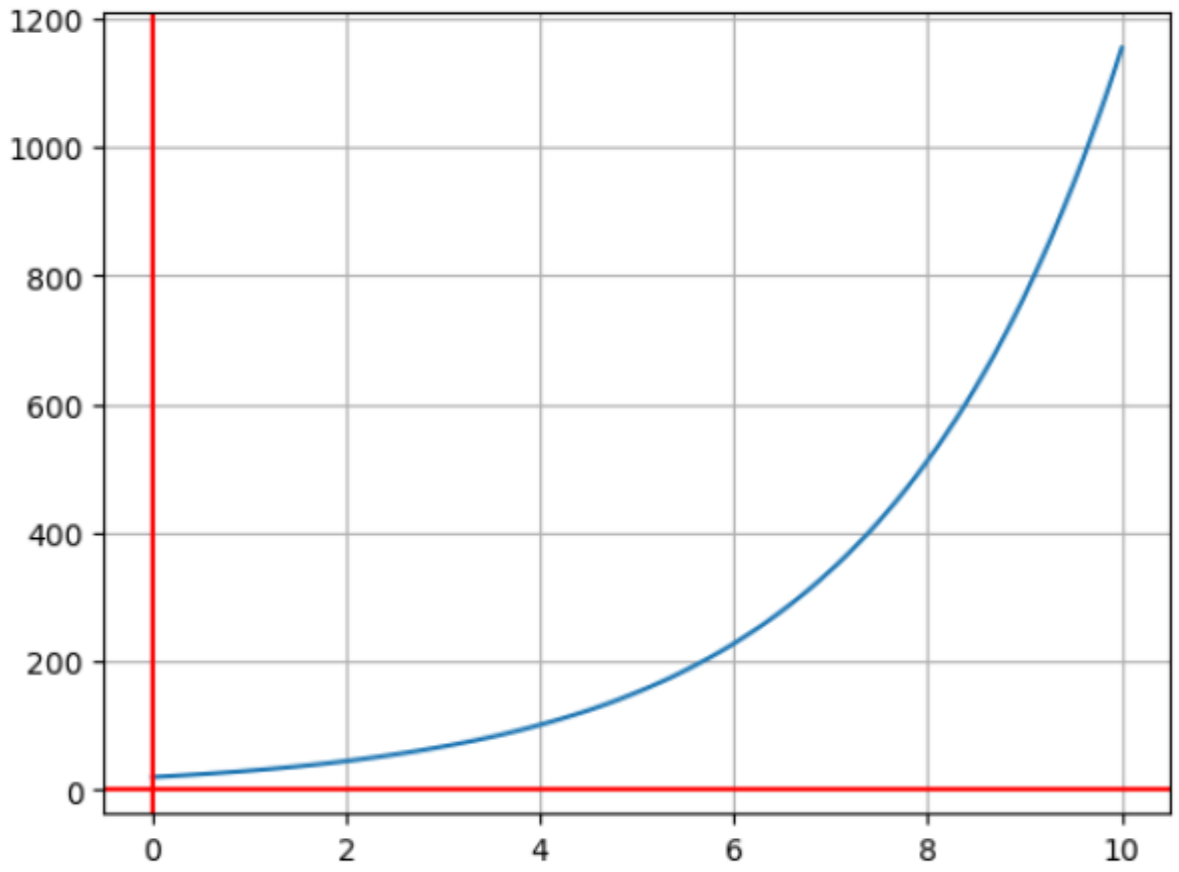
$$-kP(t) + \frac{d}{dt}P(t) = 0$$

Particular Solution is:

$$P(t) = P_0 e^{kt}$$

Hence Solution is:

$$P(t) = 20e^{0.4054651081081 64t}$$



Exercise: Write python program for the following

1. Solve:

a) $\frac{dy}{dx} = x$ b) $x^2 y' = y \log y - y'$ c) $\frac{dy}{dx} + y \tan x - y^3 \sec x = 0$

d) $x^3 \frac{dy}{dx} - x^2 y + y^4 \cos x = 0$

2. Solve:

a) $\frac{dy}{dx} = \frac{y\left(2xy + e^x\right)}{e^x}$, $y(0) = 10$

b) $y' = 4y + 2x - 4x^2$, $y(0) = 1$

c) $y' + y + y^2\left(\sin x - \cos x\right) = 0$, $y(0) = -1$

d) $\frac{dy}{dx} = x + y$, $y(2) = 0$

e) $\frac{dy}{dx} = x^2$, $y(5) = 0$

3. A body in air at 25 ^{o}C cools from 100 ^{o}C to 75 ^{o}C in 1 minute. Determine the temperature of the body after t minutes and plot the solution. Also, find the temperature of the body at the end of 3 minutes.

4. A bottle of mineral water at a room temperature of 72 ^{o}F is kept in a refrigerator where the temperature is 44 ^{o}F. After half an hour, water cooled to 61 ^{o}F. What is the temperature of the mineral water in another half an hour?

5. If number of virus affected files in a system is proportional to the number of virus affected files $P(t)$ at time t, determine the number of virus affected files at time t. Assume that the number virus affected files in the system is 1000 to begin with and after $t=1$hour the number virus affected files found to be 20,000. Plot the solution.

6. If the rate of growth of bacteria is proportional to the number of bacteria $P(t)$ present at time t, determine the number of bacteria at time t. Assume that the culture initially has 20 bacteria and after $t=1$ hour the number of bacteria measured to be 200. Plot the solution.

OBSERVATIONS

LAB 6: Solution of second-order ordinary differential equation and plotting the solution curve

Second-order linear ordinary differential equation:

If x is the independent variable and y is the dependent variable, then the standard second-order linear ordinary differential equation is

$$\frac{d^2y}{dx^2} + P(x)\frac{dy}{dx} + Q(x)y = f(x)$$

where $P(x)$, $Q(x)$ and $f(x)$ are functions of x. Above equation is called homogeneous differential equation, if $f(x) = 0$, else non-homogeneous.

Program 1: Program to solve $\frac{d^2y}{dx^2} - 5\frac{dy}{dx} + 6y = \cos 4x$

```
from sympy import *
x=symbols("x")
y=Function("y")(x)
y1=Derivative(y,x)
y2=Derivative(y1,x)
DE=Eq(y2-5*y1+6*y,cos(4*x))
print("Differential equation is: ")
display(DE)
GS=dsolve(DE,y)
print("General solution is: ")
display(GS)
```

Output:

```
Differential equation is:
```

$$6y(x) - 5\frac{d}{dx}y(x) + \frac{d^2}{dx^2}y(x) = \cos(4x)$$

```
General solution is:
```

$$y(x) = C_1e^{2x} + C_2e^{3x} - \frac{\sin(4x)}{25} - \frac{\cos(4x)}{50}$$

Program 2: Program to solve $\frac{d^2y}{dx^2}+a^2y=\sec ax,$ where a is real and $a>0$.

```
from sympy import *
a=symbols("a",real=True,positive=True)
x=symbols("x")
y=Function("y")(x)
y1=Derivative(y,x)
y2=Derivative(y1,x)
DE=Eq(y2+a**2*y,sec(a*x))
print("Differential equation is: ")
display(DE)
GS=dsolve(DE,y)
print("General Solution is:")
display(GS)
```

Output:

Differential equation is:

$$a^2 y(x)+\frac{d^2}{dx^2}y(x)=\sec(ax)$$

General Solution is:

$$y(x)=\left(C_1+\frac{\log(\cos(ax))}{a^2}\right)\cos(ax)+\left(C_2+\frac{x}{a}\right)\sin(ax)$$

Program 3: Program to solve $\frac{d^2y}{dx^2}+2\frac{dy}{dx}+2y=\cos 2x,\quad y(0)=0$ and $y'(0)=0$.

```
from sympy import *
x=symbols("x")
y=Function("y")(x)
y1=Derivative(y,x)
y2=Derivative(y1,x)
DE=Eq(y2+2*y1+2*y,cos(2*x))
display(DE)
print("Differential equation is: ")
GS=dsolve(DE,y)
print("General Solution is:")
display(GS)
PS=dsolve(DE,y,ics={y.subs({x:0}):0,y1.subs({x:0}):0})
print("Particular Solution is: ")
display(PS)
PS=lambdify(x,PS.rhs)
from pylab import *
x=linspace(0,10,100)
plot(x,PS(x))
axvline(color="red")
axhline(color="red")
grid()
show()
```

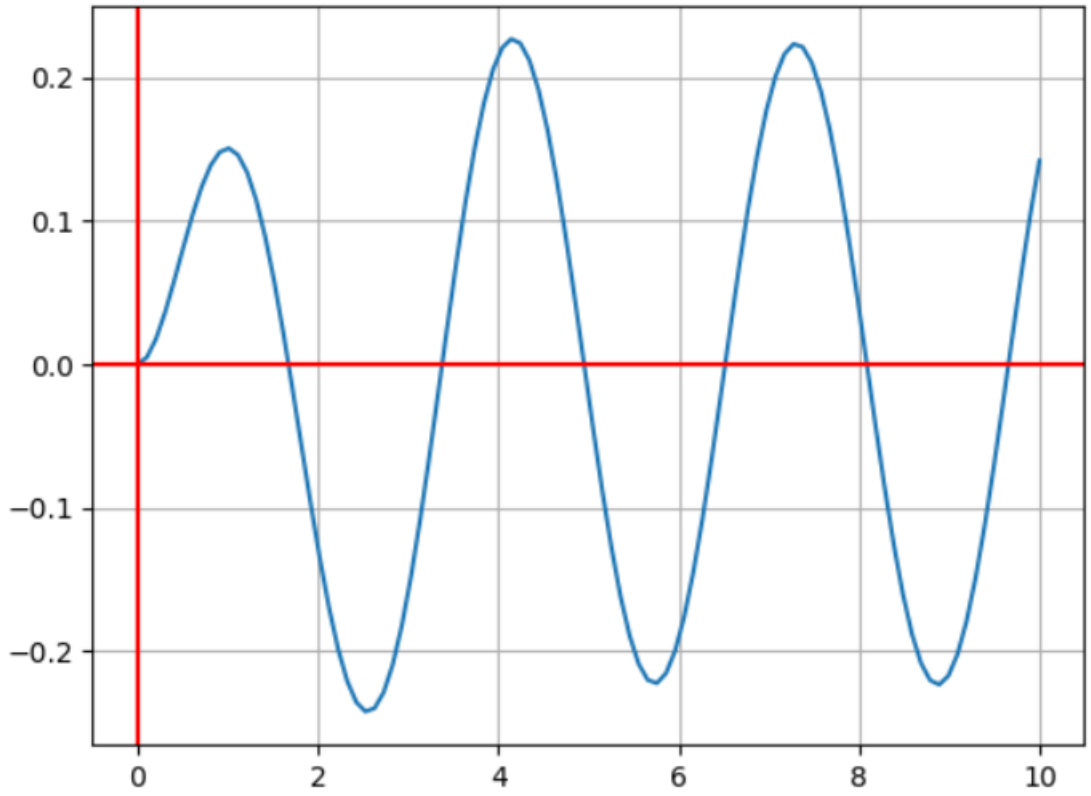
$$2y(x) + 2\frac{d}{dx}y(x) + \frac{d^2}{dx^2}y(x) = \cos(2x)$$

```
Differential equation is:
General Solution is:
```

$$y(x) = (C_1 \sin(x) + C_2 \cos(x))\, e^{-x} + \frac{\sin(2x)}{5} - \frac{\cos(2x)}{10}$$

```
Particular Solution is:
```

$$y(x) = \left(-\frac{3\sin(x)}{10} + \frac{\cos(x)}{10}\right) e^{-x} + \frac{\sin(2x)}{5} - \frac{\cos(2x)}{10}$$



Exercise: Write python program for the following

1. Solve:

a) $\frac{d^2y}{dx^2} - 4y = \cosh(2x-1) + 3^x$

b) $\frac{d^2y}{dx^2} + 2\frac{dy}{dx} + y = e^{2x} + \cos^2 x + x^2$

c) $x^2\frac{d^2y}{dx^2} - x\frac{dy}{dx} + y = \log x$

d) $\frac{d^2y}{dx^2} + 3\frac{dy}{dx} + 2y = e^{e^x}$

e) $x^2y'' - 5xy' + 6y = \cos 4\log x$

2. Solve: $r^2\frac{d^2u}{dr^2} + r\frac{du}{dr} - u = kr^3$, $u(0) = 0$ and $u'(a) = 0$

3. Solve the following and plot the solution

a) $y'' - 2y' + 10y = 0$, $y(0) = 4$ and $y'(0) = 1$

b) $\frac{d^2y}{dx^2}+4\frac{dy}{dx}+5y=-2\cosh x$, $y(0)=0$ and $y'(0)=1$

c) $3\frac{d^2y}{dx^2}+2\frac{dy}{dx}-2y=\cos 2x,\quad y(0)=0$ and $y'(0)=0$.

d) $y''+2y'+10y=e^{-x}+\sin(x)+x$, $y(0)=0$ and $y'(0)=0$

OBSERVATIONS

LAB 7: Solution of differential equation Oscillations of a spring with various load

Oscillations of a spring:

The motion of the spring-mass system is given by the differential equation (mathematical model)

$$m\frac{d^2x}{dt^2}+a\frac{dx}{dt}+kx=f(t)$$

where m is the mass of a spring coil, x is the displacement of the mass from its equilibrium position, a is damping constant and k is spring constant.

Case 1. Free and undamped motion: $a=0,\ f(t)=0$

Mathematical model: $\frac{d^2x}{dt^2}+\mu^2x=0\ ,\ \mu=\frac{k}{m}$

Case 2. Free and damped motion: $f(t)=0$

Mathematical model: $m\frac{d^2x}{dt^2}+a\frac{dx}{dt}+kx=0$

Case 3. Forced and damped motion:

Mathematical model: $m\frac{d^2x}{dt^2}+a\frac{dx}{dt}+kx=0$

Program 1: An object weighs 2 kg stretches a spring 6 m. The spring is then released from the equilibrium position with an upward velocity of 16 *m/s*. The motion of the object is denoted by $x''+\omega^2x=0$ where $\omega=8$ is the angular frequency. Find $x(t)$ using initial conditions $x(0)=0$ and $x'(0)=-16$ and plot the solution.

```
from sympy import *
t=symbols("t")
x=Function("x")(t)
x1=Derivative(x,t)
x2=Derivative(x1,t)
DE=Eq(x2+64*x,0)
display(DE)
GS=dsolve(DE,x)
print("General Solution is:")
display(GS)
PS=dsolve(DE,x,ics={x.subs({t:0}):1/4,x1.subs({t:0}):1})
print("Particular Solution is: ")
display(PS)
PS=lambdify(t,PS.rhs)
from pylab import *
t=linspace(0,10,1000)
plot(t,PS(t))
axvline(color="red")
axhline(color="red")
grid()
show()
```

Output:

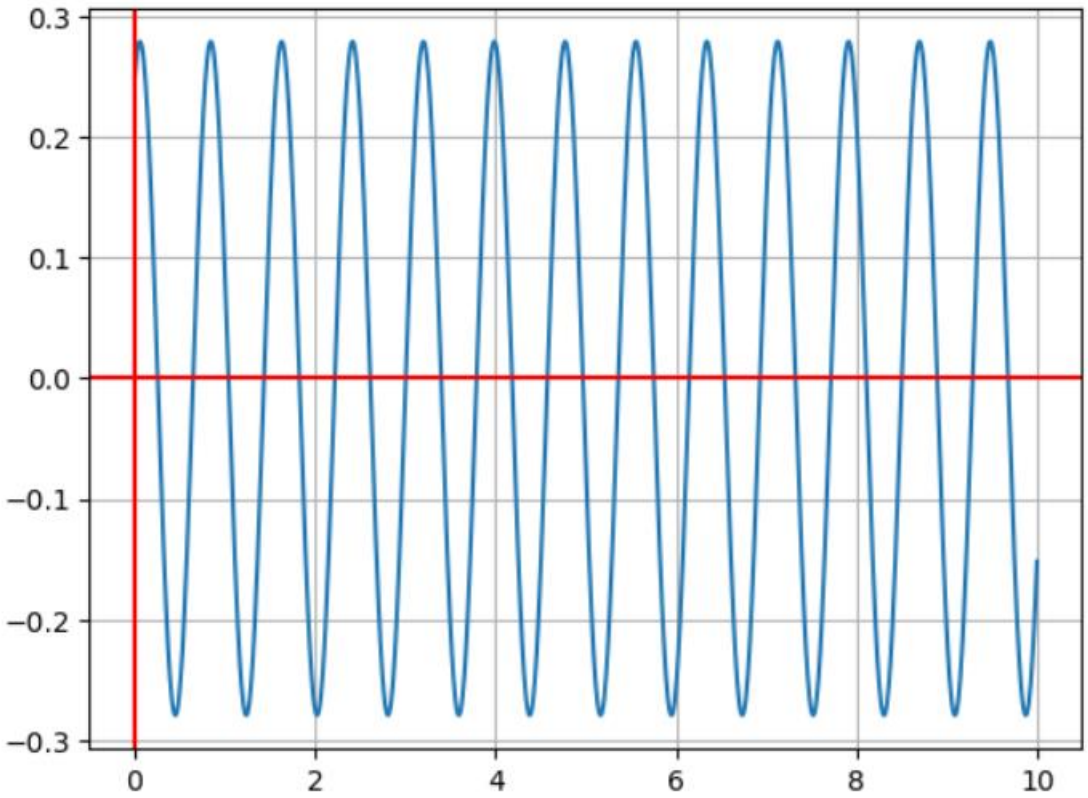
$$64x(t) + \frac{d^2}{dt^2}x(t) = 0$$

General Solution is:

$$x(t) = C_1 \sin(8t) + C_2 \cos(8t)$$

Particular Solution is:

$$x(t) = 0.125 \sin(8t) + 0.25 \cos(8t)$$



Exercise: Write python program for the following

1. The mass of 16 kg stretches a spring by $8/9$ such that there is no damping and no external forces acting on the system. The spring is initially displaced 6 inches upwards from its equilibrium position and given an initial velocity of 1 ft/sec downward. Find the displacement at any time t, $u(t)$ denoted by the second order differential equation $\frac{d^2u}{dt^2} + 36u = 0$ with initial conditions $u(0) = -\frac{1}{2}$ and $u'(0) = 1$ and plot the solution curve.

2. The instantaneous position of the base of a stamping machine is given by the solutions of the second order differential equation $y'' + 100y' = \sin 10t$. If the initial conditions are denoted by $y(0) = 0.005$ and $y'(0) = 0$, then find the position of the machine base and draw a plot for the solution.

3. Solve the following mathematical models and plot the solution

a) $x'' + 64x = 0$, $x(0) = 1/4$ and $x'(0) = 1$

b) $x'' + 6x' + 9x = \sin 3t$, $x(0) = 1$ and $x'(0) = 0$

c) $9x'' + 2x' + 1.2x = 0$, $x(0) = 1.5$ and $x'(0) = 2.5$

OBSERVATIONS

LAB 8: Determining whether the given homogeneous linear system has a non-trivial solution and to solve non homogeneous linear system if it is consistent

Reduction of a matrix to row reduced echelon form:

A matrix of order $m \times n$ is said to be in row reduced echelon form if

i) The leading element (the first non-zero entry) of each row is one.
ii) All the entries below the leading entry are zero.
iii) The zero rows must appear below the non-zero row.
iv) The number of zeros appearing before the leading entry in each row is greater than that appears in its previous row.

Rank of a matrix:

Let $'A'$ be a non-zero matrix then a positive integer $'r'$ is said to be a rank of $'A'$ if the following conditions are satisfied.

i) There exist at least one minor of order $'r'$ of matrix $'A'$ which doesn't vanish.
ii) All minors of the order $(r+1)$ must vanish.

(Or)

The rank of a matrix $'A'$ is equal to number of non-zero rows (*r*) in the matrix when it is reduced to echelon form.

It is denoted by $\rho(A) = r$.

System of linear equations:

Let us consider a system of *m* simultaneous linear algebraic equations in *n* unknowns i.e.,

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + ...a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + ...a_{2n}x_n &= b_2 \\ ..... &= ... \\ a_{m1}x_1 + a_{m2}x_2 + a_{m3}x_3 + ...a_{mn}x_n &= b_m \end{aligned}$$

Here $x_1, x_2, x_3.....x_n$ are unknowns and a_{ij}'s and b_{ij}'s are known real constants.

Above system can be written in matrix form as $AX = B$.

Where, $A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & .... & a_{1n} \\ a_{21} & a_{22} & a_{23} & .... & a_{2n} \\ .... & .... & .... & .... & .... \\ a_{m1} & a_{m2} & a_{m3} & .... & a_{mn} \end{bmatrix}$, $X = \begin{bmatrix} x_1 \\ x_2 \\ ... \\ x_n \end{bmatrix}$ & $B = \begin{bmatrix} b_1 \\ b_2 \\ ... \\ b_m \end{bmatrix}$

Further, if $B \neq 0$, we call the following matrix as augmented matrix

$$[A:B]=[A,B]=[A\,|\,B]=\begin{bmatrix} a_{11} & a_{12} & a_{13} & .... & a_{1n} & :b_1 \\ a_{21} & a_{22} & a_{23} & .... & a_{2n} & :b_2 \\ .... & .... & .... & .... & .... & :... \\ a_{m1} & a_{m2} & a_{m3} & .... & a_{mn} & :b_m \end{bmatrix}$$

System of homogeneous linear equations:

The linear system of equations of the form $AX=0$ is called system of homogeneous linear system of equations. The n - tuple $(0,0,0,...0)$ is a trivial solution of the system. The homogeneous system of m equations $AX=0$ in n unknowns has a non-trivial solution if and only if $\rho(A)$ is less than n. Further if $\rho(A)=r<n$, then the system possesses $(n-r)$ linearly independent solutions.

Non-homogeneous system of Linear Equations:

The linear system of equations of the form $AX=B$ is called system of non-homogeneous linear equations if not all elements in B are zeros. The non-homogeneous system of m equations $AX=B$ in n unknowns is

- consistent (has a solution) if and only if, $\rho(A)=\rho([A\,|\,B])$
- has unique solution if $\rho(A)=n$
- has infinitely many solutions if $\rho(A)<n$
- System is inconsistent if $\rho(A)\neq\rho([A\,|\,B])$.

Python modules used in the lab:

```
A.rank()    # Returns the rank of a matrix
Shape(A)    # Returns the shape of an array
Solve_linear_system(system,*symbols)  #Solve system of n linear equations
              with m variables, which means both under- and over determined
              systems are supported.
A.col_insert(pos, other)  # Insert one or more columns at the given column
               position.
```

Program 1: Program to check whether the following system of homogenous linear equation has non-trivial solution : $x_1+2x_2-x_3=0,\ 2x_1+x_2+4x_3=0,\ 3x_1+3x_2+4x_3=0$.

```
from sympy import *
A=Matrix([[1,2,-1],[2,1,4],[3,3,4]])
ρA=A.rank()
dim=shape(A)
print("The coefficient Matrix A is: ")
display(A)
print(f"Rank of the matrix is {ρA}")
if ρA==dim[1]:
    print("System has trivial solution")
else:
    print("System has non trivial solution")
```

Output:

The coefficient Matrix A is:

$$\begin{bmatrix} 1 & 2 & -1 \\ 2 & 1 & 4 \\ 3 & 3 & 4 \end{bmatrix}$$

```
Rank of the matrix is 3
System has trivial solution
```

Program 2: Program to solve non homogeneous linear system if it is consistent $x_1+2x_2-x_3=1,\ 2x_1+x_2+4x_3=2,\ 3x_1+3x_2+4x_3=1$.

```
from sympy import *
x,y,z=symbols("x,y,z")
A=Matrix([[1,2,-1],[2,1,4],[3,3,4]])
B=Matrix([[1],[2],[1]])
dim=shape(A)
AB=A.col_insert(dim[1],B)
ρA=A.rank()
ρAB=AB.rank()
print("The coefficient Matrix A is: ")
display(A)
print("The augmented Matrix AB is: ")
display(AB)
print(f"ρ(A)={ρA} and ρ(AB)={ρAB}")
if ρA==ρAB:
    if ρA==dim[1]:
        print("System has unique solution")
    else:
        print("The system has infinitely many solutions")
    print(solve_linear_system(AB,x,y,z))
else:
    print("The system of equations is inconsistent")
```

Output:

```
The coefficient Matrix A is:
```

$$\begin{bmatrix} 1 & 2 & -1 \\ 2 & 1 & 4 \\ 3 & 3 & 4 \end{bmatrix}$$

```
The augmented Matrix AB is:
```

$$\left[\begin{array}{cccc} 1 & 2 & -1 & 1 \\ 2 & 1 & 4 & 2 \\ 3 & 3 & 4 & 1 \end{array}\right]$$

```
ρ(A)=3 and ρ(AB)=3
System has unique solution
{x: 7, y: -4, z: -2}
```

Exercise: Write python program for the following

1. Check whether the following systems of homogenous linear equations has non-trivial solution:

a) $x_1 + 2x_2 - x_3 = 0,\ 2x_1 + x_2 + 5x_3 = 0,\ 3x_1 + 3x_2 + 4x_3 = 0$

b) $x_1 + 2x_2 - x_3 = 0,\ 2x_1 + x_2 + 4x_3 = 0,\ x_1 - x_2 + 5x_3 = 0$

c) $x + y + z = 0,\ 2x + y - 3z = 0,\ 4x - 2y - z = 0$

2. Solve the following non-homogeneous linear system if it is consistent

a) $x + y + z = 2,\ 2x + 2y - 2z = 4,\ x - 2y - z = 5$

b) $25x + y + z = 27,\ 2x + 10y - 3z = 9,\ 4x - 2y - 12z = -10$

c) $x + y + z = 2,\ 2x + 2y - 2z = 6,\ x - 2y - z = 5$

d) $2x + y + z = 4,\ 4x + 2y - 2z = 8,\ 4x + 22y + 2z = 5$

OBSERVATIONS

LAB 9: Solution of a system of linear non-homogeneous equations using the Gauss-Seidel method

Gauss-Seidel method is an iterative method to solve a system of linear equations. The method works if the system is diagonally dominant.

Diagonally dominant system:

Let us consider a system of 3 linear non-homogenous equations with 3 unknowns

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1$$
$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2$$
$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3$$

Above system is called as diagonally dominant if the diagonal co-efficients a_{11}, a_{22}, a_{33} are not zeros and $|a_{11}| > |a_{12}| + |a_{13}|$; $|a_{22}| > |a_{21}| + |a_{23}|$; $|a_{33}| > |a_{31}| + |a_{32}|$.

The working rule can be visualised through following example.

Example: Using the Gauss-Siedel method, solve the following system:

$$2x - 3y + 20z = 25 \; ; \; 20x + y - 2z = 17 \; ; \; 3x + 20y - z = -18$$

Solution: The given equations are not diagonally dominant, so rearranging it

$$\boxed{20}\,x + y - 2z = 17$$
$$3x + \boxed{20}\,y - z = -18$$
$$2x - 3y + \boxed{20}\,z = 25$$

Above equations may be rewritten as

$$20x + y - 2z = 17 \Rightarrow x = \frac{1}{20}\{17 - y + 2z\} = f_1(y, z)$$
$$3x + 20y - z = -18 \Rightarrow y = \frac{1}{20}\{-18 - 3x + z\} = f_2(x, z)$$
$$2x - 3y + 20z = 25 \Rightarrow z = \frac{1}{20}\{25 - 2x + 3y\} = f_3(x, y)$$

Let the initial approximations be $x^{(0)} = 0 \; ; y^{(0)} = 0 \; ; \; z^{(0)} = 0$

First approximation

$$\Rightarrow x^{(1)} = \frac{1}{20}\left\{17 - y^{(0)} + 2z^{(0)}\right\} = 0.8500$$

$$\Rightarrow y^{(1)} = \frac{1}{20}\left\{-18 - 3x^{(1)} + z^{(0)}\right\} = -1.0275$$

$$\Rightarrow z^{(1)} = \frac{1}{20}\left\{25 - 2x^{(1)} + 3y^{(1)}\right\} = 1.0109$$

Second approximation

$$\Rightarrow x^{(2)} = \frac{1}{20}\left\{17 - y^{(1)} + 2z^{(1)}\right\} = 1.0025$$

$$\Rightarrow y^{(2)} = \frac{1}{20}\left\{-18 - 3x^{(2)} + z^{(1)}\right\} = -0.9998$$

$$\Rightarrow z^{(2)} = \frac{1}{20}\left\{25 - 2x^{(2)} + 3y^{(2)}\right\} = 0.9998$$

Third approximation

$$\Rightarrow x^{(3)} = \frac{1}{20}\left\{17 - y^{(2)} + 2z^{(2)}\right\} = 1.0000$$

$$\Rightarrow y^{(3)} = \frac{1}{20}\left\{-18 - 3x^{(3)} + z^{(2)}\right\} = -1.0000$$

$$\Rightarrow z^{(3)} = \frac{1}{20}\left\{25 - 2x^{(3)} + 3y^{(3)}\right\} = 1.0000$$

Fourth approximation

$$\Rightarrow x^{(4)} = \frac{1}{20}\left\{17 - y^{(3)} + 2z^{(3)}\right\} = 1.0000$$

$$\Rightarrow y^{(4)} = \frac{1}{20}\left\{-18 - 3x^{(4)} + z^{(3)}\right\} = -1.0000$$

$$\Rightarrow z^{(4)} = \frac{1}{20}\left\{25 - 2x^{(4)} + 3y^{(4)}\right\} = 1.0000$$

Third and fourth approximation values are same. So $x = 1.0000,\ y = -1.0000,\ z = 1.0000$

Python modules used in the lab:

```
sys.exit()  # Exit the interpreter by raising System Exit (status).
Python sys module    # The python sys module provides functions and
              variables which are used to manipulate different parts of the
              Python Runtime Environment. It lets us access system-specific
              parameters and functions. First, we have to import the sys
              module in our program before running any functions.
```

```
lambda arguments: expression    # The expression is executed and the result is returned.
                                 A lambda function is a small anonymous Function which can take any number of arguments, but can only have one expression.
```

Program: Program to check whether the following system is diagonally dominant and if so solve it by using the Gauss-Seidel method:

$20x + y - 2z = 17;\ 3x + 20y - z = -18;\ 2x - 3y + 20z = 25$

```
from sympy import *
import sys
A=Matrix([[20,1,-2],[3,20,-1],[2,-3,20]])
B=Matrix([[17],[-18],[25]])
dim=shape(A)
for i in range(dim[1]):
    asum=0
    for j in range(dim[1]):
        if j!=i:
            asum+=abs(A[i,j])
    if asum > A[i,i]:
        sys.exit("The system is not diagonally dominant")
f1=lambda x,y,z: (B[0]-A[0,1]*y-A[0,2]*z)/A[0,0]
f2=lambda x,y,z: (B[1]-A[1,0]*x-A[1,2]*z)/A[1,1]
f3=lambda x,y,z: (B[2]-A[2,0]*x-A[2,1]*y)/A[2,2]
x0,y0,z0=0,0,0
count=1
e=0.0001   #the tolerable error
print("\nCount\t  x\t\t    y\t\t  z\n")
while True:
    x1=f1(x0,y0,z0)
    y1=f2(x1,y0,z0)
    z1=f3(x1,y1,z0)
    print("%d\t%.5f \t%.5f\t%.5f\n"%(count,x1,y1,z1))
    count=count+1
    if abs(x0-x1)<e and abs(y0-y1)<e and abs(z0-z1)<e:
        break
    else:
        x0,y0,z0=x1,y1,z1
print("\n Solution: x=%0.5f, y=%0.5f and z=%0.5f"%(x1,y1,z1))
```

Output:

Count	x	y	z
1	0.85000	-1.02750	1.01087
2	1.00246	-0.99983	0.99978
3	0.99997	-1.00001	1.00000
4	1.00000	-1.00000	1.00000

Solution: x=1.00000, y=-1.00000 and z=1.00000

Exercise: Write python program for the following

Check whether the following system is diagonally dominant and if so solve it by using Gauss-Seidel method:

a) $25x + y + z = 27;\ 2x + 10y - 3z = 9;\ 4x - 2y + 12z = -10$

b) $x + y + z = 7;\ 2x + y - 3z = 3;\ 4x - 2y - z = -1$

c) $4x + y + z = 6;\ 2x + 5y - 2z = 5;\ x - 2y - 7z = -8$

d) $27x + 6y - z = 85;\ 6x + 15y + 2z = 72;\ x + y + 54z = 110$

e) $x + 2y - z = 3;\ 3x - y + 2z = 1;\ 2x - 2y + 6z = 2$

f) $10x + y + z = 12;\ x + 10y + z = 12;\ x + y + 10z = 12$

OBSERVATIONS

LAB 10: Determining the eigenvalues and eigenvectors for the given matrix and finding the numerically largest eigenvalue by using the Rayleigh Power method

Eigenvalues and Eigenvectors:

If $A=\begin{bmatrix} a_{11} & a_{12} & a_{13} & .... & a_{1n} \\ a_{21} & a_{22} & a_{23} & .... & a_{2n} \\ .... & .... & .... & .... & .... \\ a_{m1} & a_{m2} & a_{m3} & .... & a_{mn} \end{bmatrix}$ and $X=\begin{bmatrix} x_1 \\ x_2 \\ ... \\ x_n \end{bmatrix}$ then the linear transformation $Y=AX$ carries the column vector X into column vector Y by means of a square-matrix A. It is often required to find such vectors which transform into themselves or to a scalar multiple of themselves. Let X be such a vector which transforms into λX by means of the transformation $Y=AX$.

Then $\lambda X=AX$ or $AX-\lambda IX=0$ or $\left[A-\lambda I\right]X=0$.

This represents *n* homogeneous linear equations

$$\begin{aligned} \left(a_{11}-\lambda\right)x_1+a_{12}x_2+a_{13}x_3+.....a_{1n}x_n &= 0 \\ a_{21}x_1+\left(a_{22}-\lambda\right)x_2+a_{23}x_3+.....a_{2n}x_n &= 0 \\ ..... &= ... \\ a_{n1}x_1+a_{n2}x_2+a_{n3}x_3+.....\left(a_{nn}-\lambda\right)x_n &= 0 \end{aligned}$$

Which will have a non-trivial solution only if the coefficient matrix is singular i.e., if $\left|A-\lambda I\right|=0$.

This is called the characteristic equation of the transformation and is same as the characteristic equation of the matrix A . It has *n* roots and corresponding to each root, above system will have a non-zero solution $X=\left[x_1,x_2...,x_n\right]^T$, which is known as the eigenvector.

Rayleigh's Power Method:

This is an iterative method to determine the numerically largest eigenvalue (dominant eigenvalue) & the corresponding eigenvector of a given square matrix.

Working rule:

If $'A'$ is a given square matrix

1. Consider the initial eigenvector as $X_0=\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ or $X_0=\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$ or $X_0=\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$ or $X_0=\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$

2. Compute $Y=A\,X_0$ and call the numerically largest value in Y as λ_1. So, λ_1 is the approximate eigenvalue and r $X_1=Y/\lambda_1$ is the corresponding eigenvector.

3. Now repeat step 2 iteratively until two consecutive eigenvalues are same up to a desired accuracy by assigning X_1 to X_0 in each iteration.

Python modules used in the lab:

```
w,v = linalg.eig(A) #Compute the eigenvalues, w, and right eigenvectors, v,
                      of a square array. Here, column v[:,i] is the
                      eigenvector corresponding to eigenvalue w[i]
dot(A,B)     # returns dot product of two arrays A and B
A.round(decimals=2)  #Returns A with each element rounded to the given
                      number of decimals (here 2).
```

Program 1: Program to obtain the eigenvalues and eigenvectors for the matrix

$$A=\begin{bmatrix}4 & 3 & 2\\ 1 & 4 & 1\\ 3 & 10 & 4\end{bmatrix}$$

```
1 from numpy import *
2 from sympy import Matrix
3 A=[[4,3,2],[1,4,1],[3,10,4]]
4 print("The given matrix A is")
5 display(Matrix(A))
6 eig_val,eig_vec=linalg.eig(A)
7 print("Eigen values: \n",eig_val)
8 print("Eigen values: \n",eig_vec)
```

Output:

```
The given matrix A is
```

$$\begin{bmatrix}4 & 3 & 2\\ 1 & 4 & 1\\ 3 & 10 & 4\end{bmatrix}$$

```
Eigen values:
 [8.98205672 2.12891771 0.88902557]
Eigen values:
 [[-0.49247712 -0.82039552 -0.42973429]
 [-0.26523242  0.14250681 -0.14817858]
 [-0.82892584  0.55375355  0.89071407]]
```

Program 2: Program to compute the numerically largest eigenvalue and corresponding eigenvector of the following matrix by Rayleigh - power method

$A=\begin{bmatrix} 25 & 1 & 2 \\ 1 & 3 & 0 \\ 2 & 0 & -4 \end{bmatrix}$, take $X=\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ as initial approximation to the eigenvector

```
from numpy import *
A=[[25,1,2],[1,3,0],[2,0,-4]]
X=[1,0,0]
λ_i=0
while(True):
    Y=dot(A,X)
    λ=abs(Y).max()
    X=Y/λ
    if abs(λ-λ_i)<0.001:
        break
    else:
        λ_i=λ
        print("\nThe largest Eigenvalue is: %.4f"%λ_i)
        print("\nCorresponding EigenVector is: ",X.round(decimals=4))
print("\nThe solution is:\n")
print("The largest Eigenvalue is: %.4f"%λ)
print("Corresponding EigenVector is: ",X.round(decimals=4))
```

Output:

```
The largest Eigenvalue is: 25.0000

Corresponding EigenVector is:  [1.   0.04 0.08]

The largest Eigenvalue is: 25.2000

Corresponding EigenVector is:  [1.     0.0444 0.0667]

The largest Eigenvalue is: 25.1778

Corresponding EigenVector is:  [1.     0.045  0.0688]

The largest Eigenvalue is: 25.1827

Corresponding EigenVector is:  [1.     0.0451 0.0685]

The solution is:

The largest Eigenvalue is: 25.1820
Corresponding EigenVector is:  [1.     0.0451 0.0685]
```

Exercise: Write python program for the following

1. Obtain the eigenvalues and eigenvectors of the following matrices

a) $A=\begin{bmatrix} 25 & 1 \\ 1 & 3 \end{bmatrix}$ b) $A=\begin{bmatrix} 25 & 1 & 2 \\ 1 & 3 & 0 \\ 2 & 0 & -4 \end{bmatrix}$ c) $A=\begin{bmatrix} 11 & 1 & 2 \\ 0 & 10 & 0 \\ 0 & 0 & 12 \end{bmatrix}$ d) $A=\begin{bmatrix} 3 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 12 \end{bmatrix}$

2. Find dominant eigenvalues and corresponding eigenvectors of the following matrices by using Rayleigh-power method

a) $A=\begin{bmatrix} 25 & 1 & 2 \\ 1 & 3 & 0 \\ 2 & 0 & -4 \end{bmatrix}$, take $X=\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$

b) $A=\begin{bmatrix} 6 & 1 & 2 \\ 1 & 10 & -1 \\ 2 & 1 & -4 \end{bmatrix}$, take $X=\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$

c) $A=\begin{bmatrix} 5 & 1 & 1 \\ 1 & 3 & -1 \\ 2 & -1 & -4 \end{bmatrix}$, take $X=\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$

OBSERVATIONS

Program Outcomes

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one’s own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.